

# Лекция. ViewModel и State в Compose

---

В предыдущих **сериях** практиках вы узнали о жизненном цикле действий и связанных с ним проблемах изменения конфигурации. Когда происходит изменение конфигурации, вы можете сохранить данные приложения различными способами, например, используя `rememberSaveable` или сохраняя состояние экземпляра. Однако эти варианты могут создавать проблемы. В большинстве случаев можно использовать `rememberSaveable`, но это может означать сохранение логики в `composables` или рядом с ними. Когда приложения растут, следует перемещать данные и логику подальше от составных элементов. В этом уроке вы узнаете о надежном способе разработки приложения и сохранении данных приложения при изменении конфигурации, используя преимущества библиотеки Android Jetpack, ViewModel и рекомендации по архитектуре приложений Android.

Библиотеки Android Jetpack - это набор библиотек, облегчающих разработку отличных приложений для Android. Эти библиотеки помогут вам следовать лучшим практикам, избавят вас от написания шаблонного кода и упростят сложные задачи, чтобы вы могли сосредоточиться на коде, который вам важен, например, на логике приложения.

**Архитектура приложения** - это набор правил проектирования приложения. Подобно чертежу дома, архитектура обеспечивает структуру вашего приложения. Хорошая архитектура приложения может сделать ваш код надежным, гибким, масштабируемым, тестируемым и поддерживаемым в течение многих лет. Руководство по архитектуре приложений содержит рекомендации по архитектуре приложений и рекомендуемые лучшие практики.

В этом уроке вы узнаете, как использовать `ViewModel`, один из компонентов архитектуры из библиотек Android Jetpack, который может хранить данные вашего приложения. Сохраненные данные не теряются, если фреймворк уничтожает и воссоздает действия при изменении конфигурации или других событиях. Однако данные будут потеряны, если активность будет уничтожена из-за смерти процесса.

`ViewModel` кэширует данные только при быстром воссоздании активности.

## Необходимые условия

- Знание языка Kotlin, включая функции, лямбды и композиты без статике
- Базовые знания о том, как создавать макеты в Jetpack Compose
- Базовые знания о Material Design

## Что вы узнаете

- Введение в архитектуру приложений для Android Как использовать класс `ViewModel` в своем приложении
- Как сохранить данные пользовательского интерфейса при изменении конфигурации устройства с помощью ViewModel

## Что вы создадите

Игровое приложение `Unscramble`, в котором пользователь может угадывать зашифрованные слова

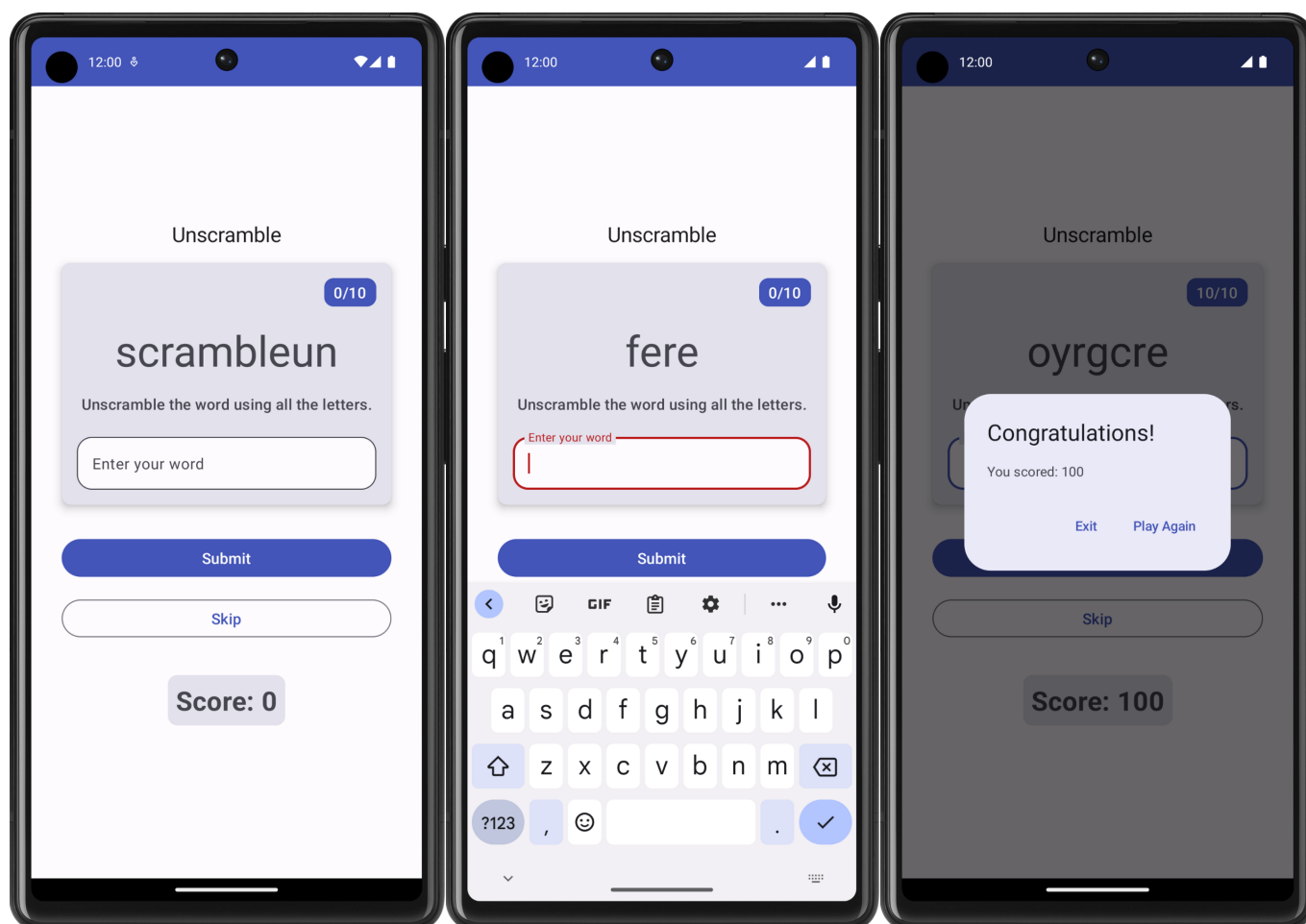
## Что вам понадобится

- Последняя версия Android Studio
- Подключение к [gogc](#) для загрузки стартового кода

## Обзор приложения

### Обзор игры

Приложение **Unscramble** - это игра-сканворд для одного игрока. Приложение показывает зашифрованное слово, а игрок должен угадать его, используя все показанные буквы. Если слово угадано правильно, игрок получает очки. В противном случае игрок может пытаться угадать слово любое количество раз. В приложении также есть возможность пропустить текущее слово. В правом верхнем углу приложение отображает количество слов - количество запутанных слов, сыгранных в текущей игре. В каждой игре 10 запутанных слов.



### Обзор стартового приложения

Чтобы ознакомиться со стартовым кодом, выполните следующие действия:

- Откройте проект со стартовым кодом в Android Studio.
- Запустите приложение на устройстве Android или в эмуляторе.
- Нажмите кнопки Submit и Skip, чтобы протестировать приложение.
- Вы заметите ошибки в приложении. Зашифрованное слово не отображается, но оно жестко закодировано на «scrambleun», и ничего не происходит, когда вы нажимаете на кнопки.

В этом уроке вы реализуете функциональность игры, используя архитектуру приложений для Android.

## Прохождение стартового кода

Стартовый код содержит заранее разработанный макет игрового экрана. На этом пути вы будете реализовывать логику игры. Вы будете использовать компоненты архитектуры для реализации рекомендуемой архитектуры приложения и решения вышеупомянутых проблем. Вот краткое описание некоторых файлов для начала работы.

- WordsData.kt

Этот файл содержит список слов, используемых в игре, константы для максимального количества слов в игре и количества очков, которые игрок получает за каждое правильное слово.

```
package com.example.android.unscramble.data

const val MAX_NO_OF_WORDS = 10
const val SCORE_INCREASE = 20

// Set with all the words for the Game
val allWords: Set<String> =
    setOf(
        "animal",
        "auto",
        "anecdote",
        "alphabet",
        "all",
        "awesome",
        "arise",
        "balloon",
        "basket",
        "bench",
        // ...
        "zoology",
        "zone",
        "zeal"
    )
```

**ПРЕДУПРЕЖДЕНИЕ:** Не рекомендуется жестко кодировать строки в коде. Добавьте строки в файл `strings.xml` для упрощения локализации. В этом примере приложения строки жестко закодированы для простоты и для того, чтобы вы могли сосредоточиться на архитектуре приложения.

- MainActivity.kt

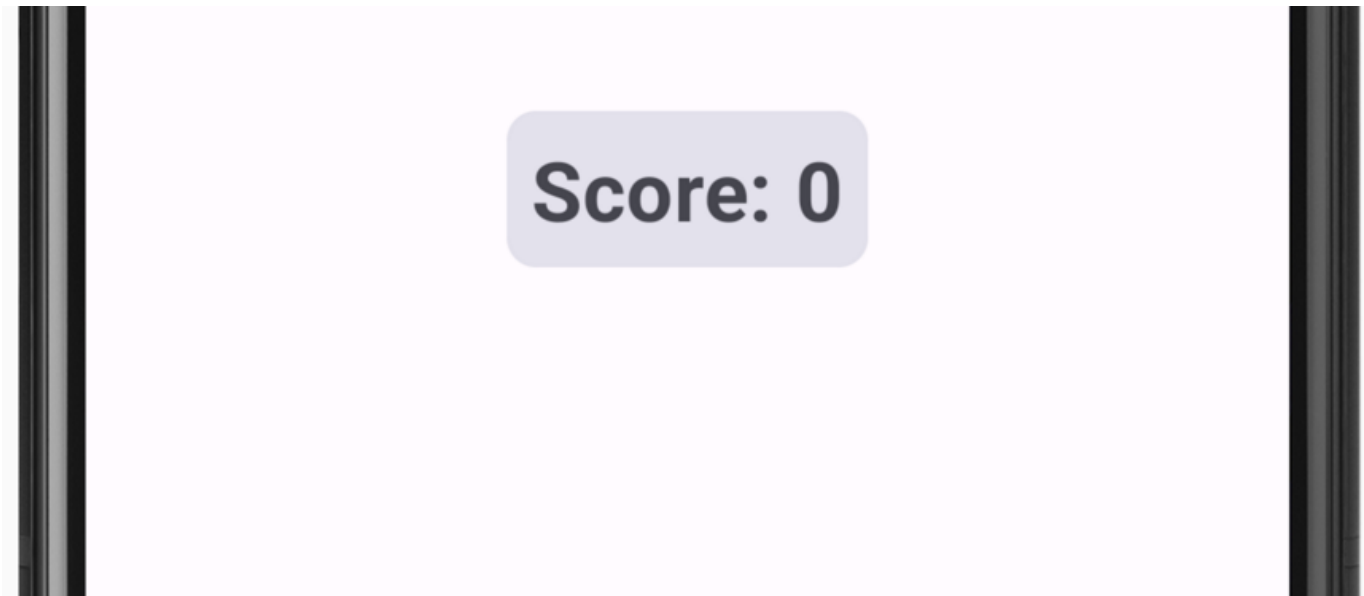
Этот файл содержит в основном код, сгенерированный по шаблону. В блоке `setContent{}` отображается композитный `GameScreen`.

- GameScreen.kt

Все составные элементы пользовательского интерфейса определяются в файле `GameScreen.kt`. В следующих разделах мы рассмотрим некоторые композитные функции.

- GameState

GameState - это композитная функция, которая отображает счет игры в нижней части экрана. Композитная функция содержит текст, компокуемый в Card. На данный момент счет жестко закодирован в 0.

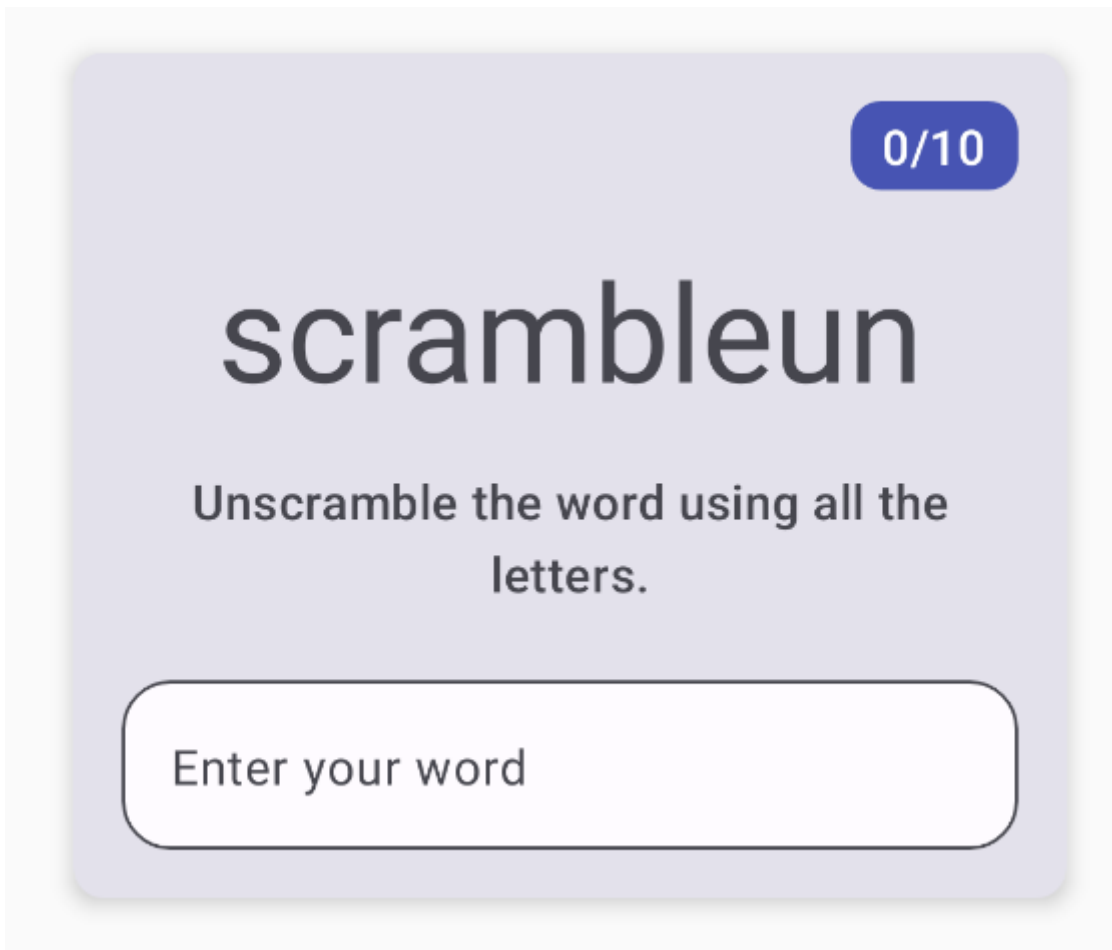


```
// No need to copy, this is included in the starter code.

@Composable
fun GameState(score: Int, modifier: Modifier = Modifier) {
    Card(
        modifier = modifier
    ) {
        Text(
            text = stringResource(R.string.score, score),
            style = typography.headlineMedium,
            modifier = Modifier.padding(8.dp)
        )
    }
}
```

- GameLayout

GameLayout - это композитная функция, которая отображает основной функционал игры, включающий в себя зашифрованное слово, инструкции по игре и текстовое поле, в котором пользователь может угадывать.



Обратите внимание, что приведенный ниже код `GameLayout` содержит колонку внутри `Card` с тремя дочерними элементами: текст зашифрованного слова, текст инструкции и текстовое поле для пользовательского слова `OutlinedTextField`. На данный момент зашифрованное слово жестко закодировано как `scrambleun`. Позже в уроке вы реализуете функциональность для отображения слова из файла `WordsData.kt`.

```
// No need to copy, this is included in the starter code.

@Composable
fun GameLayout(modifier: Modifier = Modifier) {
    val mediumPadding = dimensionResource(R.dimen.padding_medium)
    Card(
        modifier = modifier,
        elevation = CardDefaults.cardElevation(defaultElevation = 5.dp)
    ) {
        Column(
            verticalArrangement = Arrangement.spacedBy(mediumPadding),
            horizontalAlignment = Alignment.CenterHorizontally,
            modifier = Modifier.padding(mediumPadding)
        ) {
            Text(
                modifier = Modifier
                    .clip(shapes.medium)
                    .background(colorScheme.surfaceTint)
                    .padding(horizontal = 10.dp, vertical = 4.dp)
                    .align(alignment = Alignment.End),
```

```

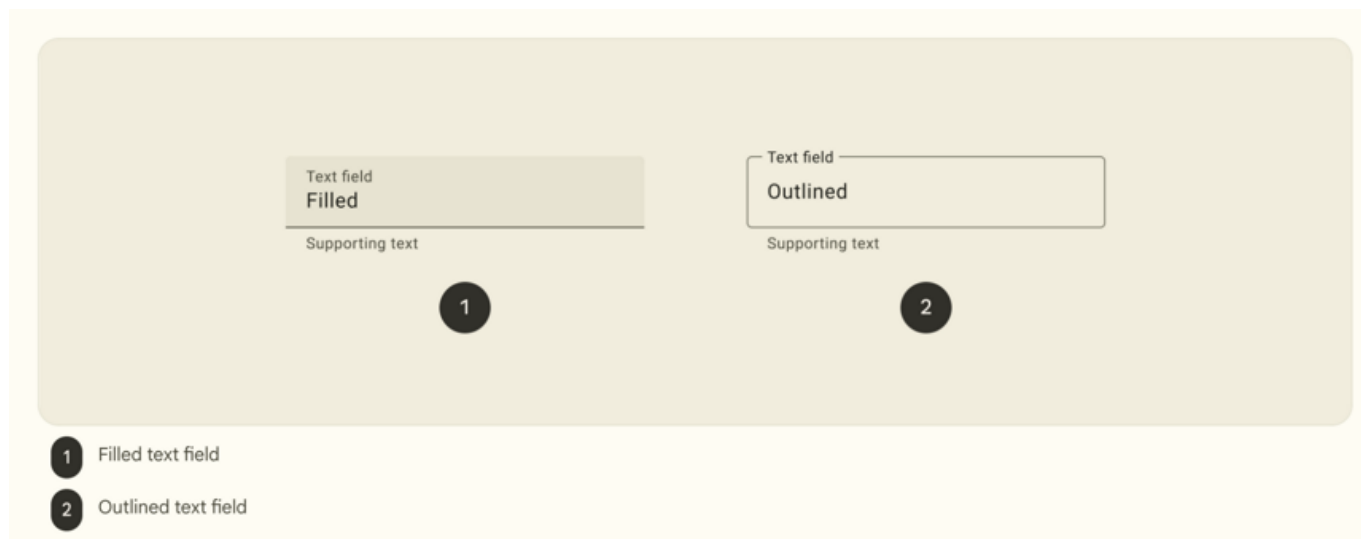
        text = stringResource(R.string.word_count, 0),
        style = typography.titleMedium,
        color = colorScheme.onPrimary
    )
    Text(
        text = "scrambleun",
        style = typography.displayMedium
    )
    Text(
        text = stringResource(R.string.instructions),
        textAlign = TextAlign.Center,
        style = typography.titleMedium
    )
    OutlinedTextField(
        value = "",
        singleline = true,
        shape = shapes.large,
        modifier = Modifier.fillMaxWidth(),
        colors = TextFieldDefaults.textFieldColors(containerColor =
colorScheme.surface),
        onValueChange = { },
        label = { Text(stringResource(R.string.enter_your_word)) },
        isError = false,
        keyboardOptions = KeyboardOptions.Default.copy(
            imeAction = ImeAction.Done
        ),
        keyboardActions = KeyboardActions(
            onDone = { }
        )
    )
}
}
}
}

```

Составное поле `OutlinedTextField` похоже на составное поле `TextField` из приложений предыдущих уроках.

Текстовые поля бывают двух типов:

- Заполненные текстовые поля
- Обведенные текстовые поля

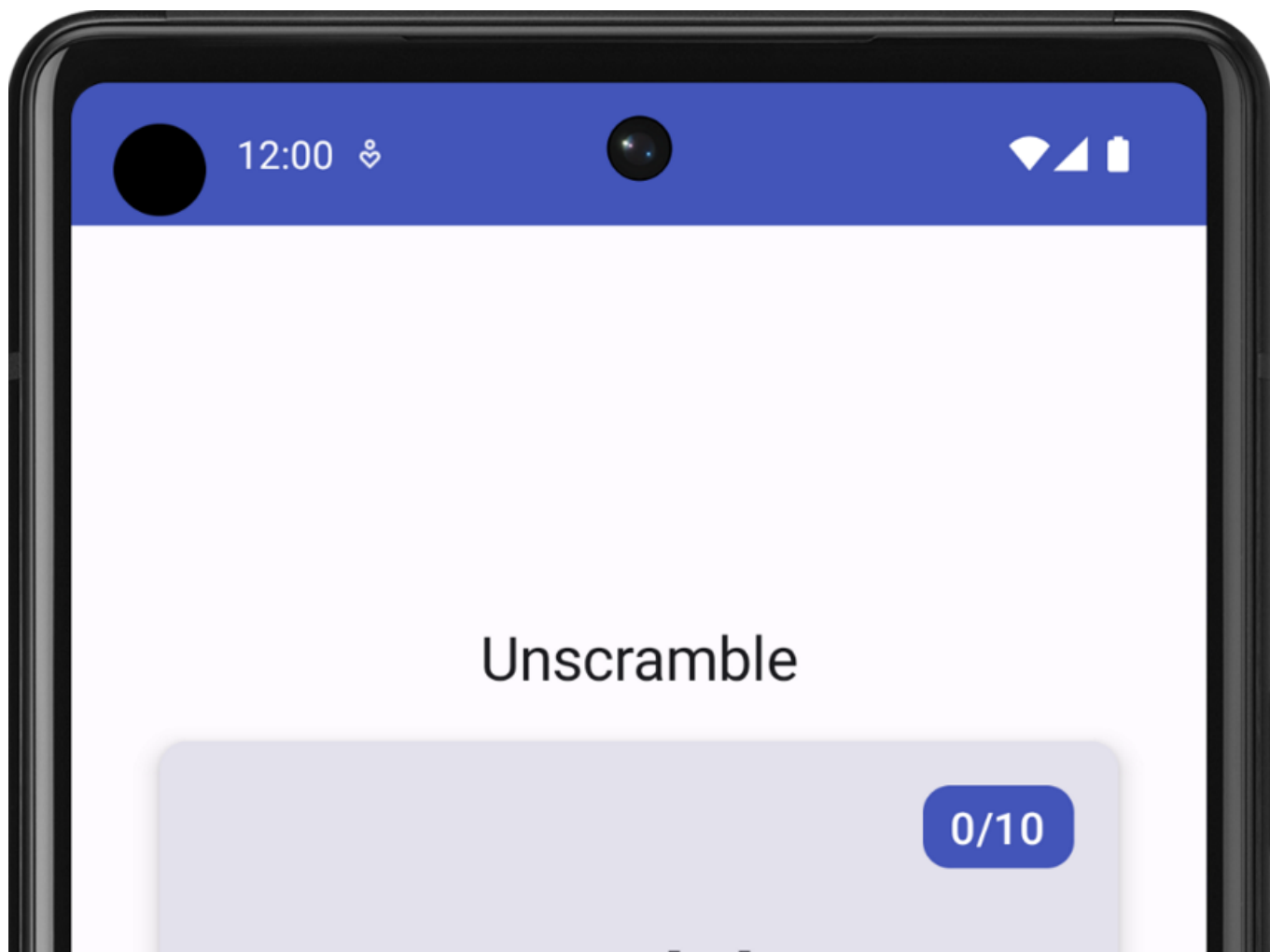


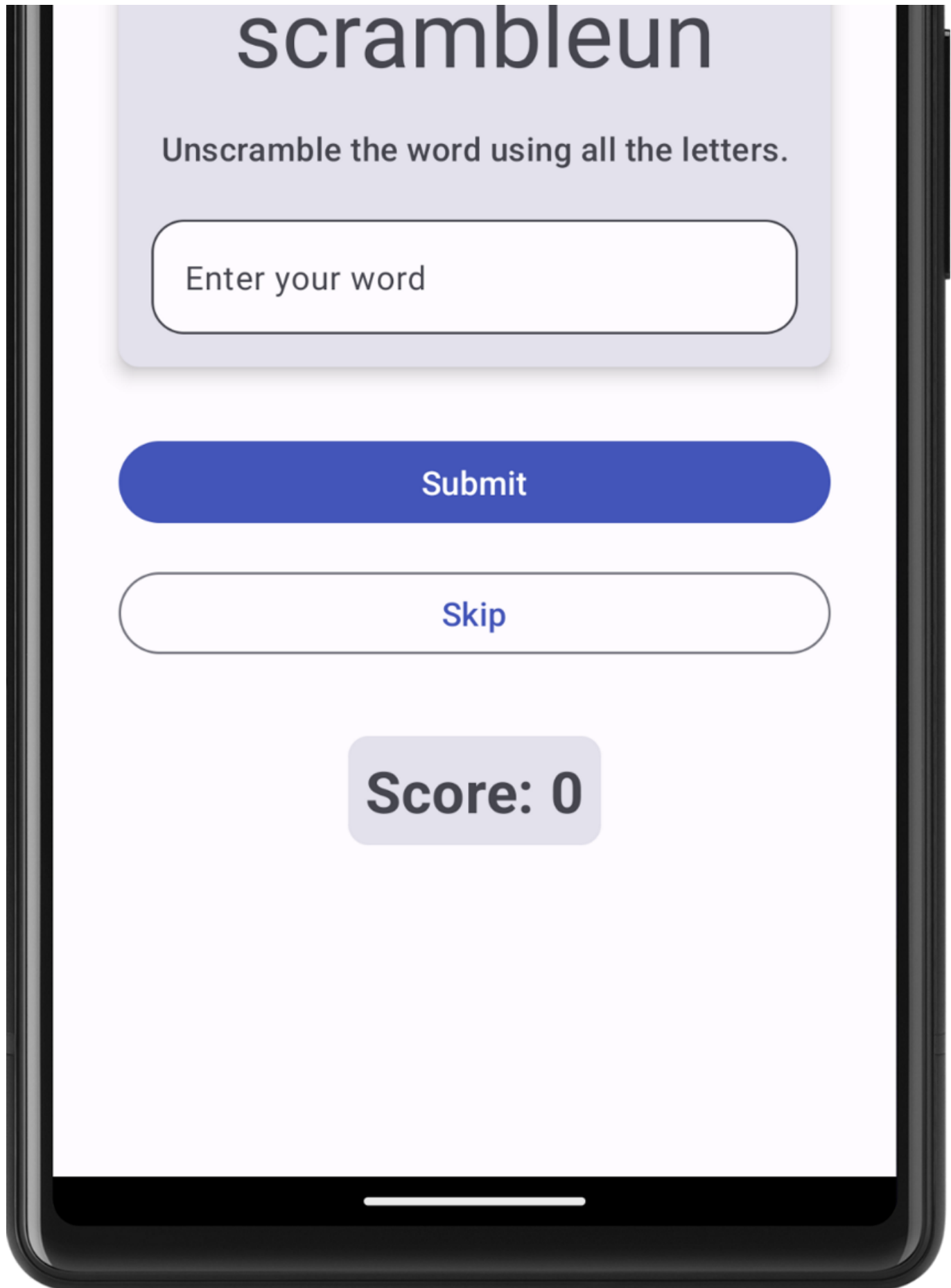
Обведенные текстовые поля имеют меньший визуальный акцент, чем заполненные текстовые поля. Когда они появляются в таких местах, как формы, где много текстовых полей расположены вместе, их уменьшенный акцент помогает упростить макет.

В исходном коде текстовое поле `OutlinedTextField` не обновляется, когда пользователь вводит угаданное значение. Вы обновите эту функцию в codelab.

## GameScreen

Составной экран `GameScreen` содержит составные функции `GameStatus` и `GameLayout`, название игры, количество слов и составные элементы для кнопок `Submit` и `Skip`.





{style='width:500px'}

```

@Composable
fun GameScreen() {
    val mediumPadding = dimensionResource(R.dimen.padding_medium)

    Column(
        modifier = Modifier
            .verticalScroll(rememberScrollState())
            .padding(mediumPadding),
        verticalArrangement = Arrangement.Center,
        horizontalAlignment = Alignment.CenterHorizontally
    ) {

        Text(
            text = stringResource(R.string.app_name),
            style = typography.titleLarge,
        )

        GameLayout(
            modifier = Modifier
                .fillMaxWidth()
                .wrapContentHeight()
                .padding(mediumPadding)
        )
        Column(
            modifier = Modifier
                .fillMaxWidth()
                .padding(mediumPadding),
            verticalArrangement = Arrangement.spacedBy(mediumPadding),
            horizontalAlignment = Alignment.CenterHorizontally
        ) {

            Button(
                modifier = Modifier.fillMaxWidth(),
                onClick = { }
            ) {
                Text(
                    text = stringResource(R.string.submit),
                    fontSize = 16.sp
                )
            }

            OutlinedButton(
                onClick = { },
                modifier = Modifier.fillMaxWidth()
            ) {
                Text(
                    text = stringResource(R.string.skip),
                    fontSize = 16.sp
                )
            }
        }

        GameStatus(score = 0, modifier = Modifier.padding(20.dp))
    }
}

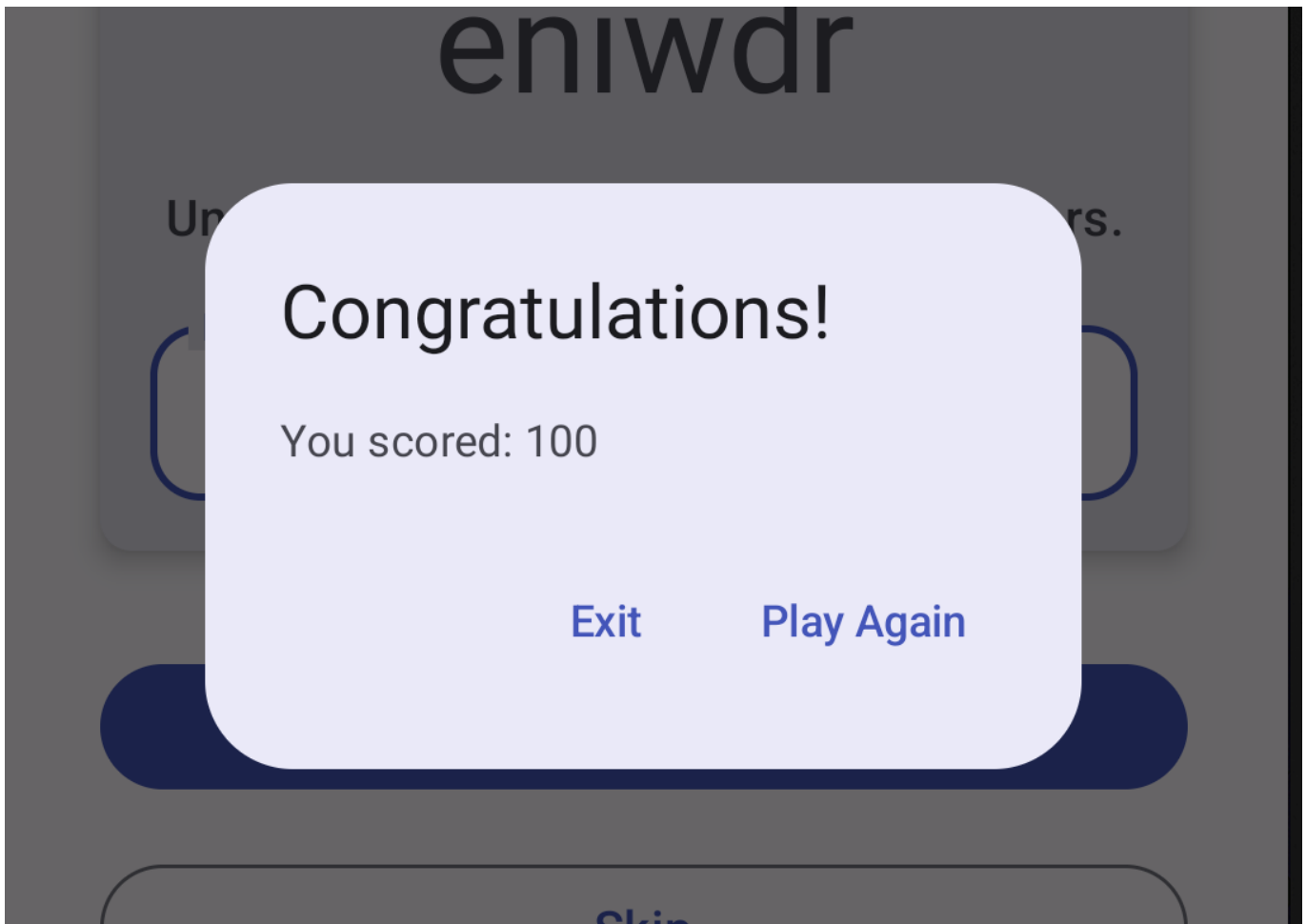
```

```
}  
}
```

События нажатия на кнопку не реализованы в начальном коде. Вы будете реализовывать эти события в рамках урока.

- `FinalScoreDialog`

Композит `FinalScoreDialog` отображает диалог, то есть небольшое окно, в котором пользователю предлагается выбрать опции Play Again или Exit the game. Позже в этом уроке вы реализуете логику для отображения этого диалога в конце игры.



{style="width:400px"}

```
// No need to copy, this is included in the starter code.
```

```
@Composable  
private fun FinalScoreDialog(  
    score: Int,  
    onPlayAgain: () -> Unit,  
    modifier: Modifier = Modifier  
) {  
    val activity = (LocalContext.current as Activity)  
  
    AlertDialog(  
        onDismissRequest = {
```

```

        // Dismiss the dialog when the user clicks outside the dialog or on the
back        // button. If you want to disable that functionality, simply use an
empty        // onDismissRequest.
    },
    title = { Text(text = stringResource(R.string.congratulations)) },
    text = { Text(text = stringResource(R.string.you_scored, score)) },
    modifier = modifier,
    dismissButton = {
        TextButton(
            onClick = {
                activity.finish()
            }
        ) {
            Text(text = stringResource(R.string.exit))
        }
    },
    confirmButton = {
        TextButton(onClick = onPlayAgain) {
            Text(text = stringResource(R.string.play_again))
        }
    }
)
}

```

## Узнайте об архитектуре приложения

**Архитектура** приложения содержит рекомендации, которые помогут вам распределить обязанности приложения между классами. Хорошо продуманная архитектура приложения помогает масштабировать приложение и расширять его за счет дополнительных функций. Архитектура также может упростить совместную работу команды.

Наиболее распространенными архитектурными принципами являются разделение проблем и управление пользовательским интерфейсом на основе модели.

### Разделение обязанностей

Принцип разделения задач гласит, что приложение делится на классы функций, каждый из которых несет отдельную ответственность.

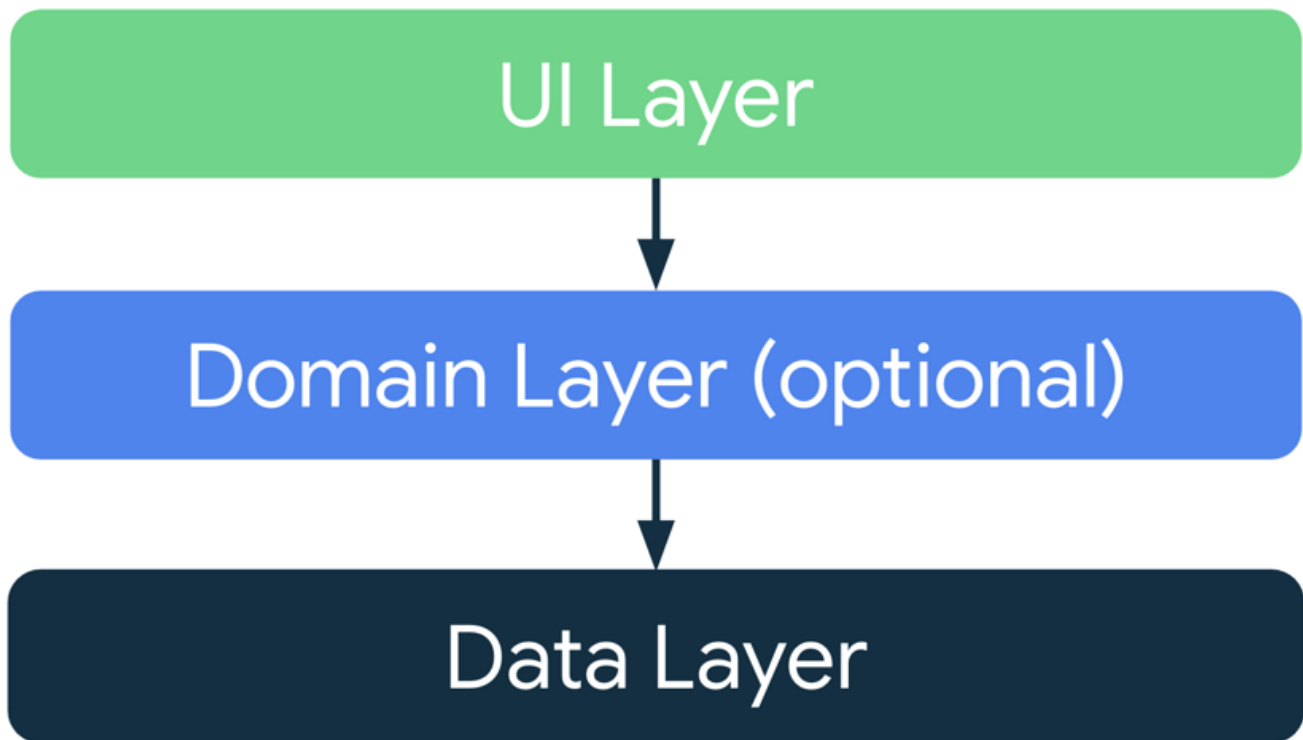
### Управление пользовательским интерфейсом на основе модели

Принцип управления пользовательским интерфейсом от модели гласит, что пользовательский интерфейс должен управляться от модели, предпочтительно постоянной модели. **Модели** - это компоненты, отвечающие за обработку данных в приложении. Они независимы от элементов пользовательского интерфейса и компонентов приложения, поэтому на них не влияет жизненный цикл приложения и связанные с ним проблемы.

## Рекомендуемая архитектура приложений

Учитывая общие архитектурные принципы, упомянутые в предыдущем разделе, каждое приложение должно иметь как минимум два слоя:

**Слой пользовательского интерфейса:** слой, который отображает данные приложения на экране, но не зависит от данных. **Слой данных:** слой, который хранит, извлекает и раскрывает данные приложения. Вы можете добавить еще один слой, называемый **слоем домена**, чтобы упростить и повторно использовать взаимодействия между слоями пользовательского интерфейса и данных. Этот слой является необязательным и выходит за рамки данного курса.



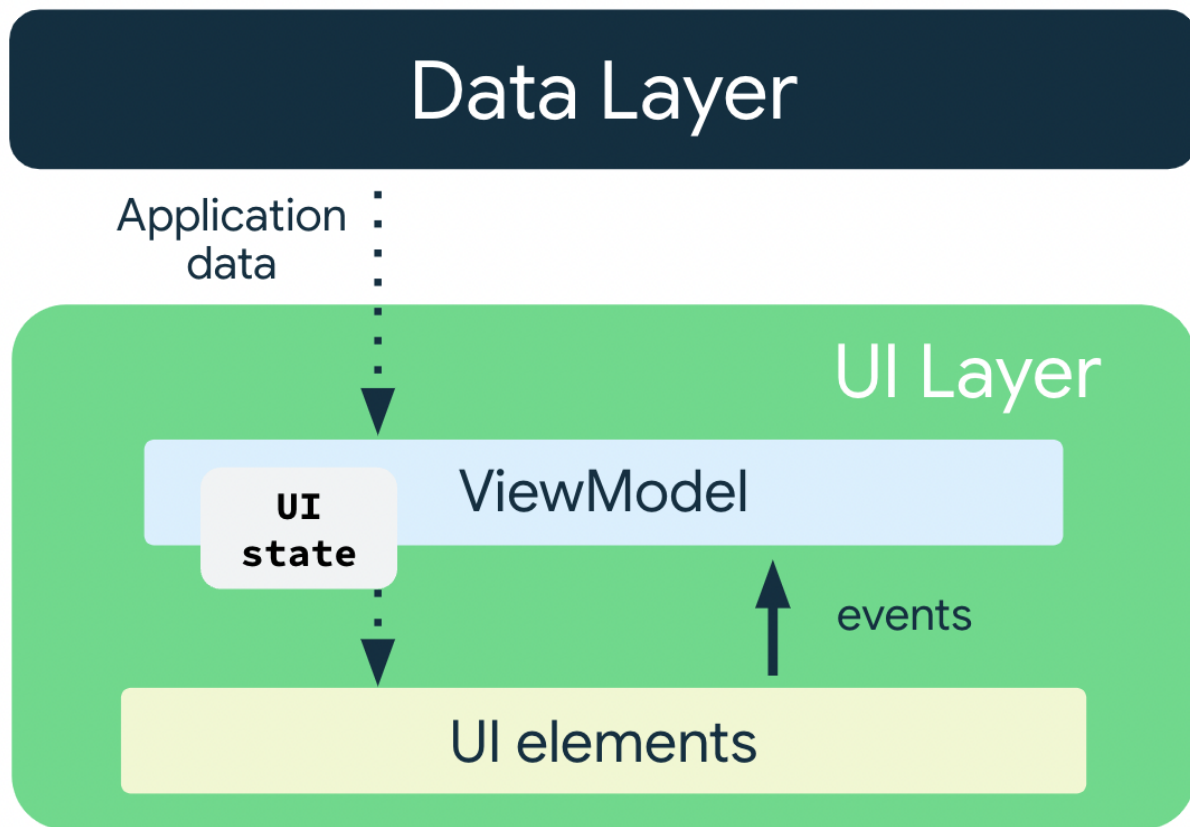
Примечание: Стрелки на диаграммах в этом руководстве представляют зависимости между классами. Например, слой домена зависит от классов слоя данных.

## Слой пользовательского интерфейса

Роль слоя пользовательского интерфейса, или презентационного слоя, заключается в отображении данных приложения на экране. Когда данные изменяются в результате взаимодействия с пользователем, например нажатия кнопки, пользовательский интерфейс должен обновляться, чтобы отразить эти изменения.

Слой пользовательского интерфейса состоит из следующих компонентов:

**Элементы пользовательского интерфейса:** компоненты, которые отображают данные на экране. Вы создаете эти элементы с помощью Jetpack Compose. **Держатели состояний:** компоненты, которые хранят данные, отображают их в пользовательском интерфейсе и обрабатывают логику приложения. Примером держателя состояния является `ViewModel`.



## ViewModel

Компонент ViewModel хранит и раскрывает состояние, потребляемое пользовательским интерфейсом. Состояние пользовательского интерфейса - это данные приложения, преобразованные **ViewModel**. ViewModel позволяет вашему приложению следовать архитектурному принципу управления пользовательским интерфейсом из модели.

**ViewModel** хранит данные, связанные с приложением, которые не уничтожаются при уничтожении и воссоздании активности фреймворком Android. В отличие от экземпляра активности, объекты ViewModel не уничтожаются. Приложение автоматически сохраняет объекты ViewModel при изменении конфигурации, чтобы данные, которые они хранят, были сразу доступны после рекомпозиции.

- Чтобы реализовать ViewModel в своем приложении, расширьте класс ViewModel, который поставляется из библиотеки архитектурных компонентов и хранит данные приложения в этом классе.

## Состояние пользовательского интерфейса

Пользовательский интерфейс - это то, что видит пользователь, а состояние пользовательского интерфейса - это то, что, по мнению приложения, он должен видеть. UI - это визуальное представление состояния UI. Любые изменения в состоянии пользовательского интерфейса немедленно отражаются в пользовательском интерфейсе.



Пользовательский интерфейс - это результат связывания элементов пользовательского интерфейса на экране с состоянием пользовательского интерфейса.

```
// Example of UI state definition, do not copy over
```

```
data class NewsItemUiState(  
    val title: String,  
    val body: String,  
    val bookmarked: Boolean = false,  
    ...  
)
```

**Неизменяемость** Определение состояния пользовательского интерфейса в приведенном выше примере является неизменяемым. Неизменяемые объекты гарантируют, что множество источников не изменят состояние приложения в определенный момент времени. Такая защита позволяет пользовательскому интерфейсу сосредоточиться на единственной роли: чтении состояния и соответствующем обновлении элементов пользовательского интерфейса. Поэтому никогда не следует изменять состояние пользовательского интерфейса непосредственно в нем, если только сам пользовательский интерфейс не является единственным источником своих данных. Нарушение этого принципа приводит к появлению нескольких источников истины для одного и того же фрагмента информации, что приводит к несогласованности данных и мелким ошибкам.

## Добавьте ViewModel

В этом задании вы добавляете в приложение модель ViewModel для хранения состояния пользовательского интерфейса игры (зашифрованное слово, количество слов и счет). Чтобы решить проблему в стартовом коде, которую вы заметили в предыдущем разделе, вам нужно сохранить данные игры в ViewModel.

- Откройте `build.gradle.kts` (Модуль :app), прокрутите до блока зависимостей и добавьте следующую зависимость для ViewModel. Эта зависимость используется для добавления в приложение - `lifecycle:lifecycle-viewmodel-compose`.

```
dependencies {  
    // other dependencies  
  
    implementation("androidx.lifecycle:lifecycle-viewmodel-compose:2.6.1")
```

```
//...  
}
```

- В пакете `ui` создайте класс/файл Kotlin с именем `GameViewModel`. Расширьте его из класса `ViewModel`.

```
import androidx.lifecycle.ViewModel  
  
class GameViewModel : ViewModel() {  
}
```

В пакете `ui` добавьте класс модели для состояния UI под названием `GameUiState`. Сделайте его классом данных и добавьте переменную для текущего зашифрованного слова.

```
data class GameUiState(  
    val currentScrambledWord: String = ""  
)
```

## StateFlow

`StateFlow` - это наблюдаемый поток с держателем данных, который выдает обновления текущего и нового состояния. Его свойство `value` отражает текущее значение состояния. Чтобы обновить состояние и отправить его в поток, присвойте новое значение свойству `value` класса `MutableStateFlow`.

В Android `StateFlow` хорошо работает с классами, которые должны поддерживать наблюдаемое неизменяемое состояние.

Поток `StateFlow` может быть открыт из `GameUiState`, чтобы композиты могли прослушивать обновления состояния пользовательского интерфейса и заставлять состояние экрана выдерживать изменения конфигурации.

В класс `GameViewModel` добавьте следующее свойство `_uiState`.

```
import kotlinx.coroutines.flow.MutableStateFlow  
  
// Game UI state  
private val _uiState = MutableStateFlow(GameUiState())
```

## Свойство Backing

Свойство `backing` позволяет вам вернуть что-то из геттера, а не сам объект.

Для свойства `var` фреймворк Kotlin генерирует геттеры и сеттеры.

Для методов `getter` и `setter` вы можете переопределить один или оба метода и обеспечить собственное поведение. Для реализации свойства `backing` вы переопределяете метод `getter`, чтобы вернуть версию данных, доступную только для чтения. В следующем примере показано подкрепляющее свойство:

```
//Example code, no need to copy over

// Declare private mutable variable that can only be modified
// within the class it is declared.
private var _count = 0

// Declare another public immutable field and override its getter method.
// Return the private property's value in the getter method.
// When count is accessed, the get() function is called and
// the value of _count is returned.
val count: Int
    get() = _count
```

В качестве другого примера, скажем, что вы хотите, чтобы данные приложения были приватными для `ViewModel`:

Внутри класса `ViewModel`:

Свойство `_count` является приватным и изменяемым. Следовательно, оно доступно и редактируемо только в классе `ViewModel`.

За пределами класса `ViewModel`:

Модификатором видимости по умолчанию в Kotlin является `public`, поэтому `count` является `public` и доступен из других классов, например из контроллеров пользовательского интерфейса. Тип `val` не может иметь сеттера. Он неизменяем и доступен только для чтения, поэтому вы можете переопределить только метод `get()`. Когда внешний класс обращается к этому свойству, он возвращает значение `_count`, и его значение не может быть изменено. Это резервное свойство защищает данные приложения внутри `ViewModel` от нежелательных и небезопасных изменений со стороны внешних классов, но позволяет внешним пользователям безопасно получать доступ к его значению.

- В файле `GameViewModel.kt` добавьте резервное свойство `uiState` с именем `_uiState`. Назовите свойство `uiState`, и оно будет иметь тип `StateFlow<GameUiState>`.

Теперь `_uiState` доступно и редактируется только внутри `GameViewModel`. Пользовательский интерфейс может прочитать его значение, используя свойство `uiState`, доступное только для чтения. Ошибку инициализации можно исправить в следующем шаге.

```
import kotlinx.coroutines.flow.StateFlow

// Game UI state

// Backing property to avoid state updates from other classes
```

```
private val _uiState = MutableStateFlow(GameUiState())
val uiState: StateFlow<GameUiState>
```

- Установите uiState в \_uiState.asStateFlow(). Функция asStateFlow() превращает этот мутабельный поток состояний в поток состояний, доступный только для чтения.

```
import kotlinx.coroutines.flow.StateFlow
import kotlinx.coroutines.flow.asStateFlow

// Game UI state
private val _uiState = MutableStateFlow(GameUiState())
val uiState: StateFlow<GameUiState> = _uiState.asStateFlow()
```

## Отображение случайного зашифрованного слова

В этом задании вы добавите вспомогательные методы для выбора случайного слова из `WordsData.kt` и скремблирования этого слова.

- В GameViewModel добавьте свойство currentWord типа String для сохранения текущего зашифрованного слова.

```
private lateinit var currentWord: String
```

- Добавьте вспомогательный метод, чтобы выбрать случайное слово из списка и перетасовать его. Назовите его pickRandomWordAndShuffle() без входных параметров и сделайте так, чтобы он возвращал String.

```
import com.example.unscramble.data.allWords

private fun pickRandomWordAndShuffle(): String {
    // Continue picking up a new random word until you get one that hasn't been
    used before
    currentWord = allWords.random()
    if (usedWords.contains(currentWord)) {
        return pickRandomWordAndShuffle()
    } else {
        usedWords.add(currentWord)
        return shuffleCurrentWord(currentWord)
    }
}
```

Android Studio выдает ошибку из-за неопределенной переменной и функции.

- В GameViewModel добавьте следующее свойство после свойства currentWord, которое будет служить в качестве изменяемого набора для хранения используемых в игре слов.

```
// Set of words used in the game
private var usedWords: MutableSet<String> = mutableSetOf()
```

- Добавьте еще один вспомогательный метод для перетасовки текущего слова под названием `shuffleCurrentWord()`, который принимает строку и возвращает перетасованную строку.

```
private fun shuffleCurrentWord(word: String): String {
    val tempWord = word.toCharArray()
    // Scramble the word
    tempWord.shuffle()
    while (String(tempWord).equals(word)) {
        tempWord.shuffle()
    }
    return String(tempWord)
}
```

- Добавьте вспомогательную функцию для инициализации игры под названием `resetGame()`. Позже вы будете использовать эту функцию для запуска и перезапуска игры. В этой функции очистите все слова в наборе `usedWords`, инициализируйте `_uiState`. Выберите новое слово для `currentScrambledWord` с помощью функции `pickRandomWordAndShuffle()`.

```
fun resetGame() {
    usedWords.clear()
    _uiState.value = GameUiState(currentScrambledWord = pickRandomWordAndShuffle())
}
```

- Добавьте блок `init` к `GameViewModel` и вызовите из него `resetGame()`.

```
init {
    resetGame()
}
```

Когда вы создаете свое приложение сейчас, вы по-прежнему не видите никаких изменений в пользовательском интерфейсе. Вы не передаете данные из `ViewModel` в композиты на `GameScreen`.

## Архитектура пользовательского интерфейса Compose

В `Compose` единственный способ обновить пользовательский интерфейс - это изменить состояние приложения.

Все, что вы можете контролировать, - это состояние пользовательского интерфейса. Каждый раз, когда состояние пользовательского интерфейса меняется, `Compose` воссоздает те части дерева пользовательского интерфейса, которые изменились. Составные элементы могут принимать состояние

и отображать события. Например, TextField/OutlinedTextField принимает значение и выставляет обратный вызов onValueChange, который запрашивает обработчик обратного вызова на изменение значения.

```
//Example code no need to copy over

var name by remember { mutableStateOf("") }
OutlinedTextField(
    value = name,
    onValueChange = { name = it },
    label = { Text("Name") }
)
```

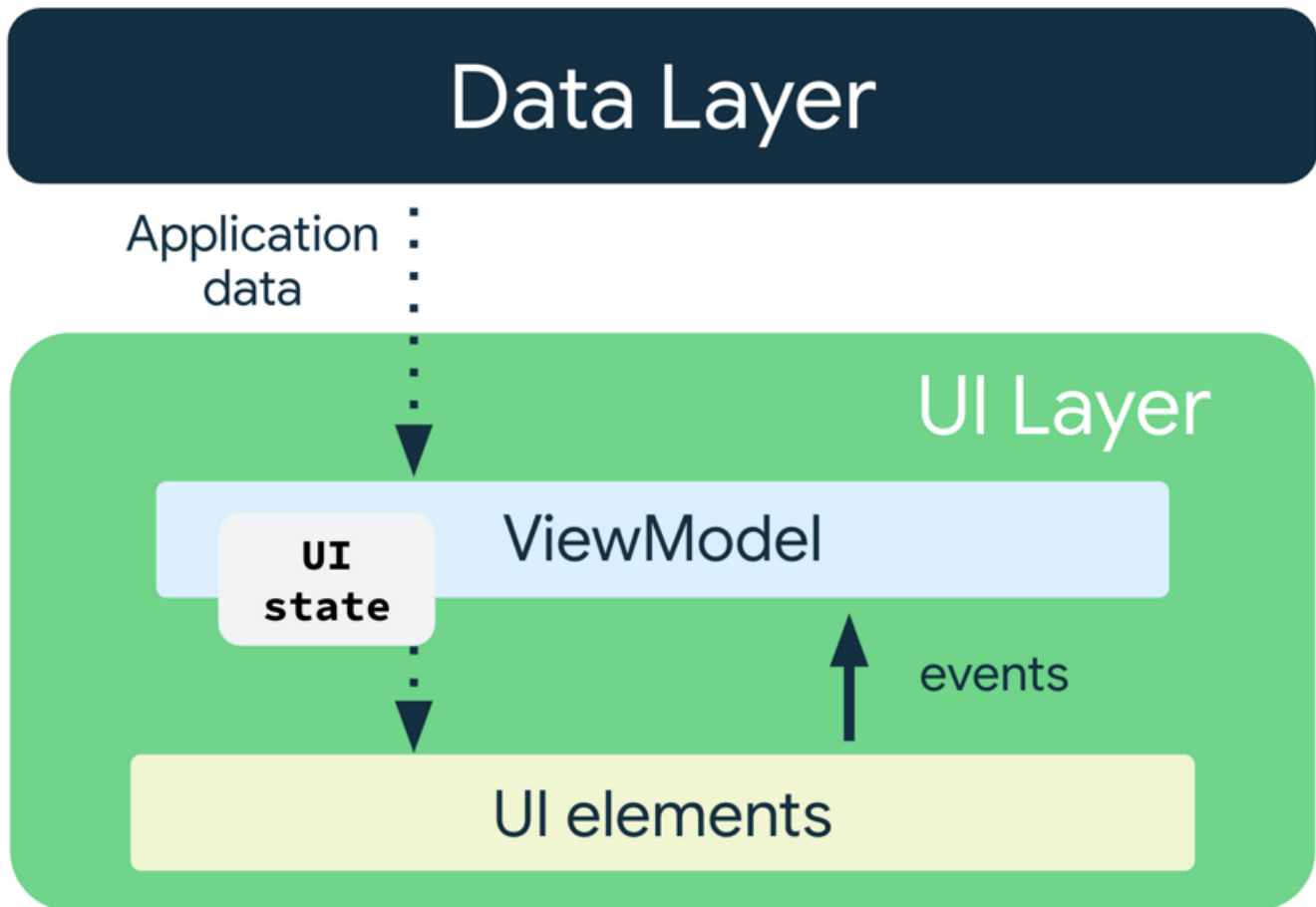
Поскольку композиты принимают события состояния и отображения, шаблон однонаправленного потока данных хорошо подходит для Jetpack Compose. В этом разделе мы рассмотрим, как реализовать шаблон однонаправленного потока данных в Compose, как реализовать события и держатели состояния, а также как работать с ViewModels в Compose.

## Однонаправленный поток данных

Однонаправленный поток данных (UDF) - это паттерн проектирования, в котором состояние течет вниз, а события - вверх. Следуя однонаправленному потоку данных, вы можете разделить строительные блоки, отображающие состояние в пользовательском интерфейсе, и части вашего приложения, которые хранят и изменяют состояние.

Цикл обновления пользовательского интерфейса в приложении, использующем однонаправленный поток данных, выглядит следующим образом:

**Событие:** Часть пользовательского интерфейса генерирует событие и передает его наверх - например, нажатие кнопки, переданное ViewModel для обработки, или событие, переданное из других слоев приложения, например, указание на то, что пользовательская сессия истекла. **Обновление состояния:** обработчик события может изменить состояние события. **Отображение состояния:** Обработчик состояния передает состояние, и пользовательский интерфейс отображает его.



Использование паттерна UDF в архитектуре приложений имеет следующие последствия:

- ViewModel хранит и раскрывает состояние, потребляемое пользовательским интерфейсом.
- Состояние пользовательского интерфейса - это данные приложения, преобразованные ViewModel.
- Пользовательский интерфейс уведомляет ViewModel о событиях пользователя.
- ViewModel обрабатывает действия пользователя и обновляет состояние.
- Обновленное состояние передается обратно в пользовательский интерфейс для рендеринга.

Этот процесс повторяется для любого события, вызывающего изменение состояния.

### Передача данных

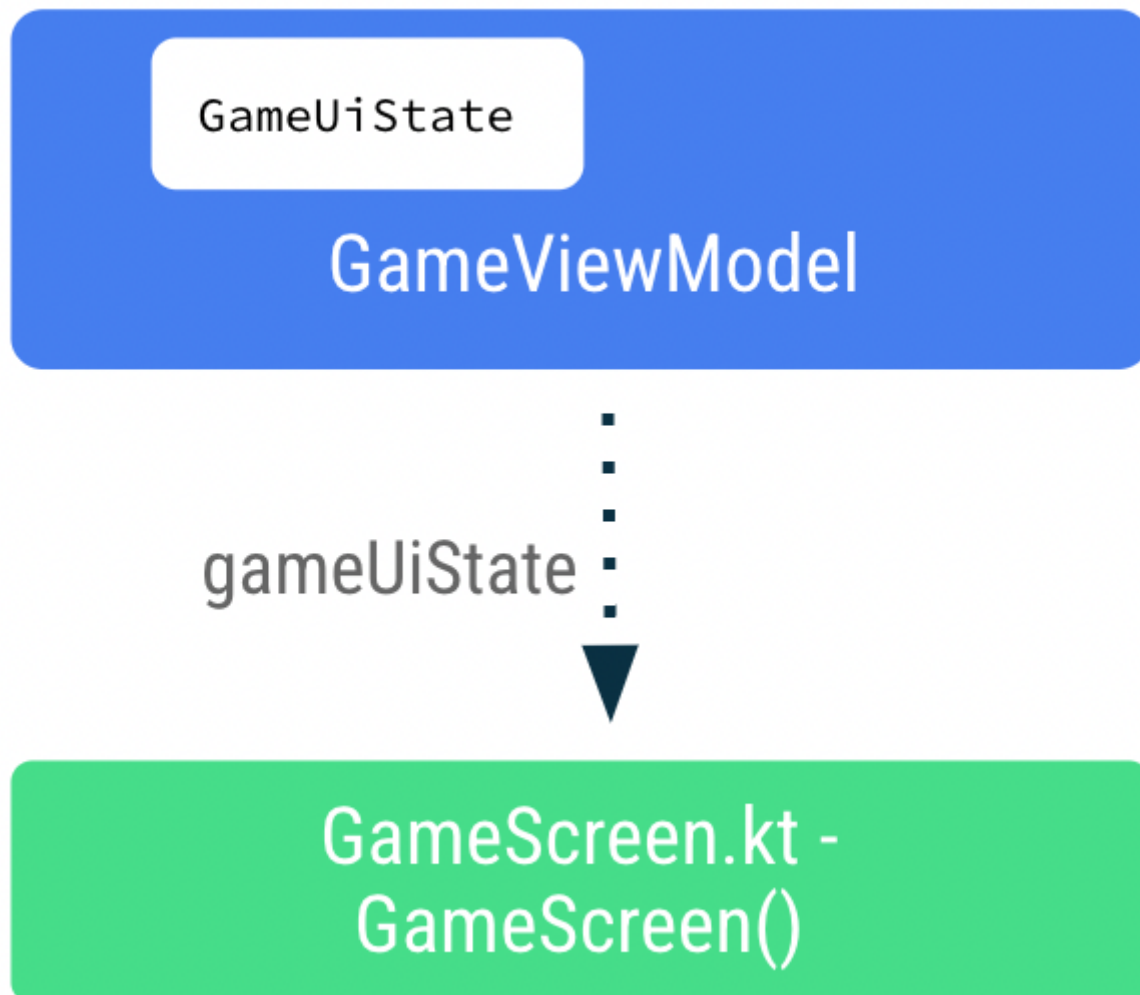
- Передайте экземпляр ViewModel в пользовательский интерфейс - то есть из GameViewModel в GameScreen() в файле GameScreen.kt. В GameScreen() используйте экземпляр ViewModel для доступа к uiState с помощью функции collectAsState().

Функция collectAsState() собирает значения из данного StateFlow и представляет его последнее значение через State. StateFlow.value используется в качестве начального значения. Каждый раз, когда в StateFlow будет помещено новое значение, возвращаемое State обновляется, что приводит к перекомпозиции каждого использования State.value.

- В функции GameScreen передайте второй аргумент типа GameViewModel со значением по умолчанию viewModel().

```
import androidx.lifecycle.viewmodel.compose.viewModel

@Composable
fun GameScreen(
    gameViewModel: GameViewModel = viewModel()
) {
    // ...
}
```



- В функции `GameScreen()` добавьте новую переменную `gameUiState`. Используйте делегат `by` и вызовите функцию `collectAsState()` для `uiState`.

Такой подход гарантирует, что при каждом изменении значения `uiState` произойдет перекomпозиция композиций, использующих значение `gameUiState`.

```
import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue

@Composable
fun GameScreen(
    // ...
}
```

```

    ) {
        val gameUiState by gameViewModel.uiState.collectAsState()
        // ...
    }

```

```

import androidx.compose.runtime.collectAsState
import androidx.compose.runtime.getValue

@Composable
fun GameScreen(
    // ...
) {
    val gameUiState by gameViewModel.uiState.collectAsState()
    // ...
}

```

- Передайте `gameUiState.currentScrambledWord` в составной `GameLayout()`. Вы добавите этот аргумент на следующем шаге, поэтому пока проигнорируйте ошибку.

```

GameLayout(
    currentScrambledWord = gameUiState.currentScrambledWord,
    modifier = Modifier
        .fillMaxWidth()
        .wrapContentHeight()
        .padding(mediumPadding)
)

```

- Добавьте `currentScrambledWord` в качестве еще одного параметра в составную функцию `GameLayout()`.

```

@Composable
fun GameLayout(
    currentScrambledWord: String,
    modifier: Modifier = Modifier
) {
}

```

- Обновите составную функцию `GameLayout()`, чтобы отобразить `currentScrambledWord`. Установите параметр `text` первого текстового поля в колонке на `currentScrambledWord`.

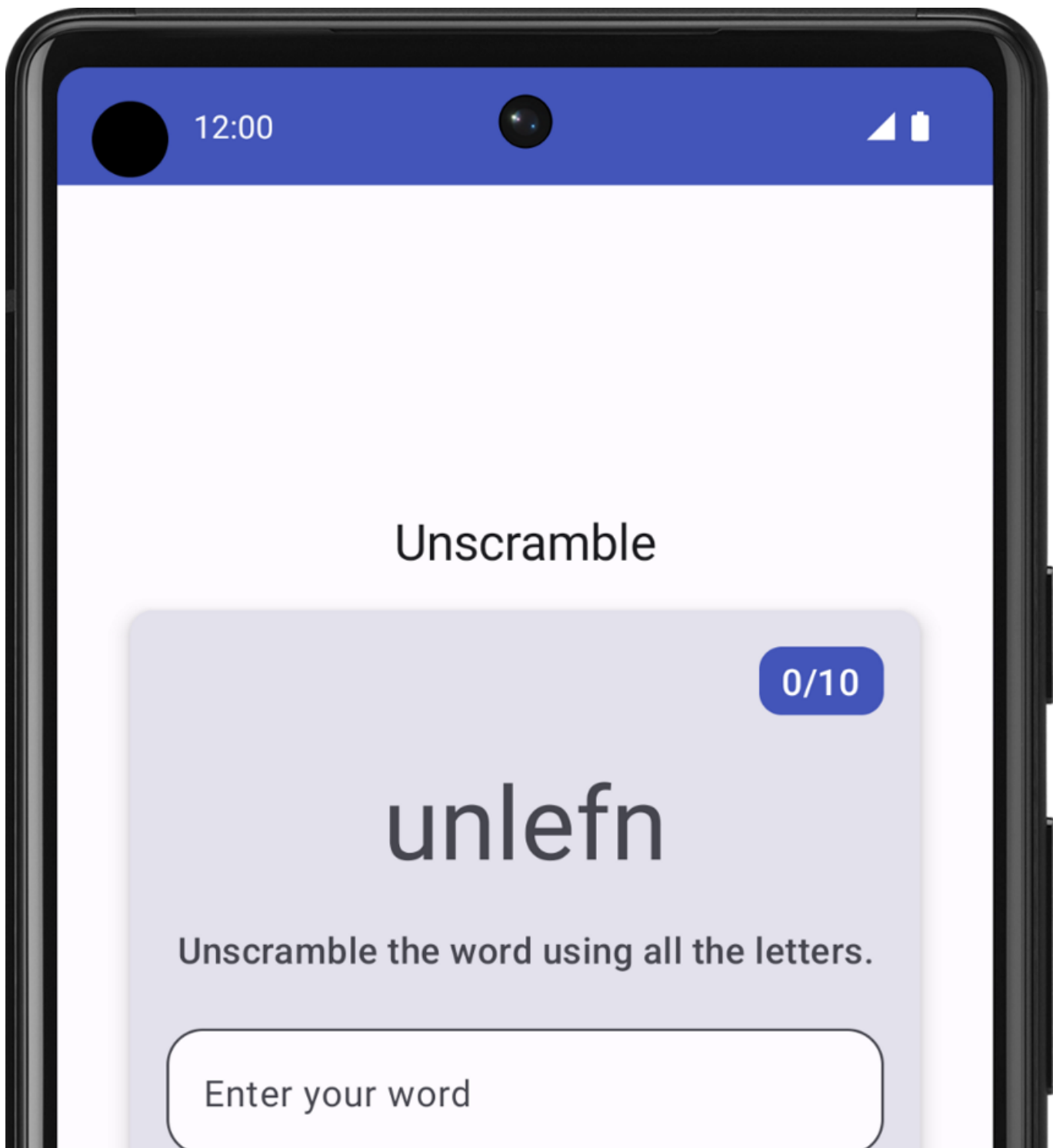
```

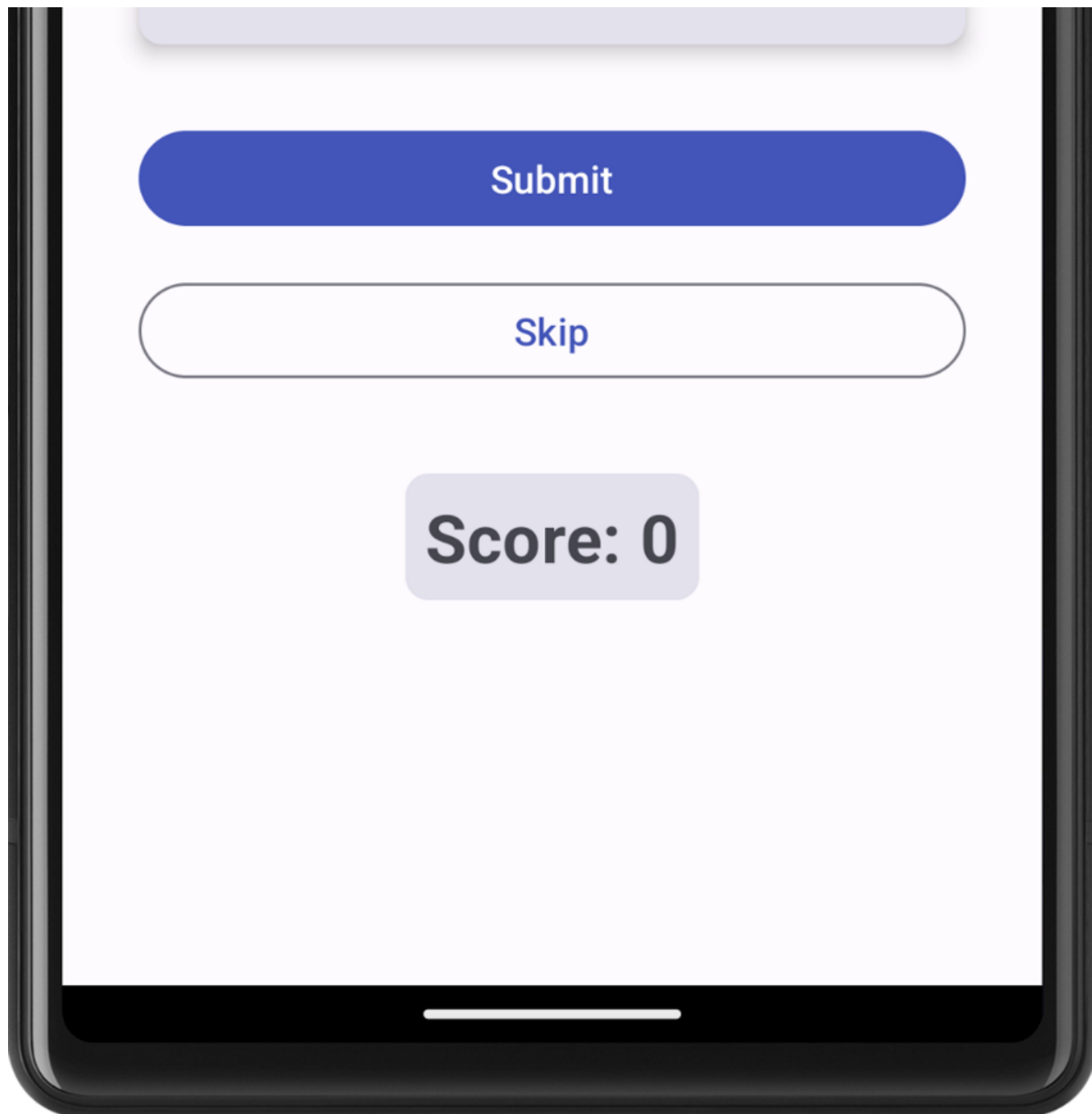
@Composable
fun GameLayout(
    // ...
) {

```

```
Column(  
    verticalArrangement = Arrangement.spacedBy(24.dp)  
) {  
    Text(  
        text = currentScrambledWord,  
        fontSize = 45.sp,  
        modifier = modifier.align(Alignment.CenterHorizontally)  
    )  
    //...  
}
```

- Запустите и создайте приложение. Вы должны увидеть зашифрованное слово.

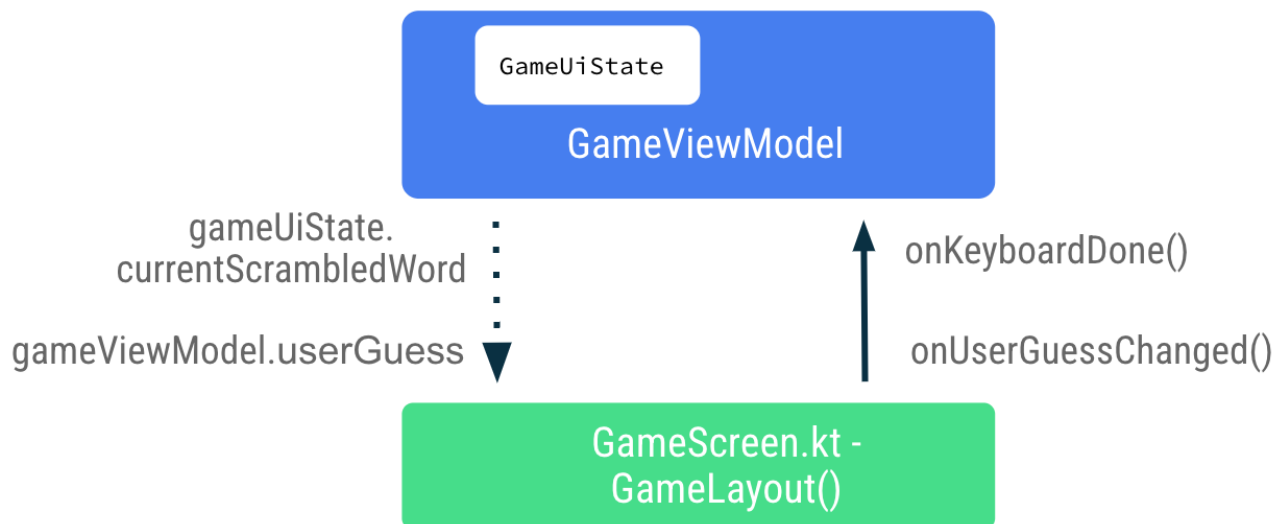




{style="width:500px"}

### Отображение угаданного слова

В композите `GameLayout()` обновление угаданного пользователем слова является одним из обратных вызовов событий, которые передаются от `GameScreen` к `ViewModel`. Данные `gameViewModel.userGuess` будут передаваться вниз от `ViewModel` к `GameScreen`.



- В файле `GameScreen.kt` в композите `GameLayout()` установите `onValueChange` на `onUserGuessChanged`, а `onKeyboardDone()` на `onDone` keyboard action. Исправьте ошибки в следующем шаге.

```
OutlinedTextField(
    value = "",
    singleLine = true,
    modifier = Modifier.fillMaxWidth(),
    onValueChange = onUserGuessChanged,
    label = { Text(stringResource(R.string.enter_your_word)) },
    isError = false,
    keyboardOptions = KeyboardOptions.Default.copy(
        imeAction = ImeAction.Done
    ),
    keyboardActions = KeyboardActions(
        onDone = { onKeyboardDone() }
    ),
)
```

- В составной функции `GameLayout()` добавьте еще два аргумента: лямбда `onUserGuessChanged` принимает аргумент `String` и ничего не возвращает, а `onKeyboardDone` ничего не принимает и не возвращает.

```
@Composable
fun GameLayout(
    onUserGuessChanged: (String) -> Unit,
    onKeyboardDone: () -> Unit,
    currentScrambledWord: String,
    modifier: Modifier = Modifier,
) {
}
```

- В вызове функции `GameLayout()` добавьте лямбда-аргументы для `onUserGuessChanged` и `onKeyboardDone`.

```
GameLayout(  
    onUserGuessChanged = { gameViewModel.updateUserGuess(it) },  
    onKeyboardDone = { },  
    currentScrambledWord = gameUiState.currentScrambledWord,  
)
```

Вскоре вы определите метод `updateUserGuess` в `GameViewModel`.

- В файле `GameViewModel.kt` добавьте метод `updateUserGuess()`, который принимает аргумент `String` - угаданное пользователем слово. Внутри функции обновите `userGuess` переданным угаданным словом.

```
fun updateUserGuess(guessedWord: String){  
    userGuess = guessedWord  
}
```

Далее вы добавляете `userGuess` во `ViewModel`.

- В файле `GameViewModel.kt` добавьте свойство `var` под названием `userGuess`. Используйте `mutableStateOf()`, чтобы `Compose` наблюдал за этим значением и устанавливал начальное значение в «».

```
import androidx.compose.runtime.mutableStateOf  
import androidx.compose.runtime.getValue  
import androidx.compose.runtime.setValue  
  
var userGuess by mutableStateOf("")  
private set
```

- В файле `GameScreen.kt`, внутри `GameLayout()`, добавьте еще один параметр `String` для `userGuess`. Установите параметр `value` поля `OutlinedTextField` в значение `userGuess`.

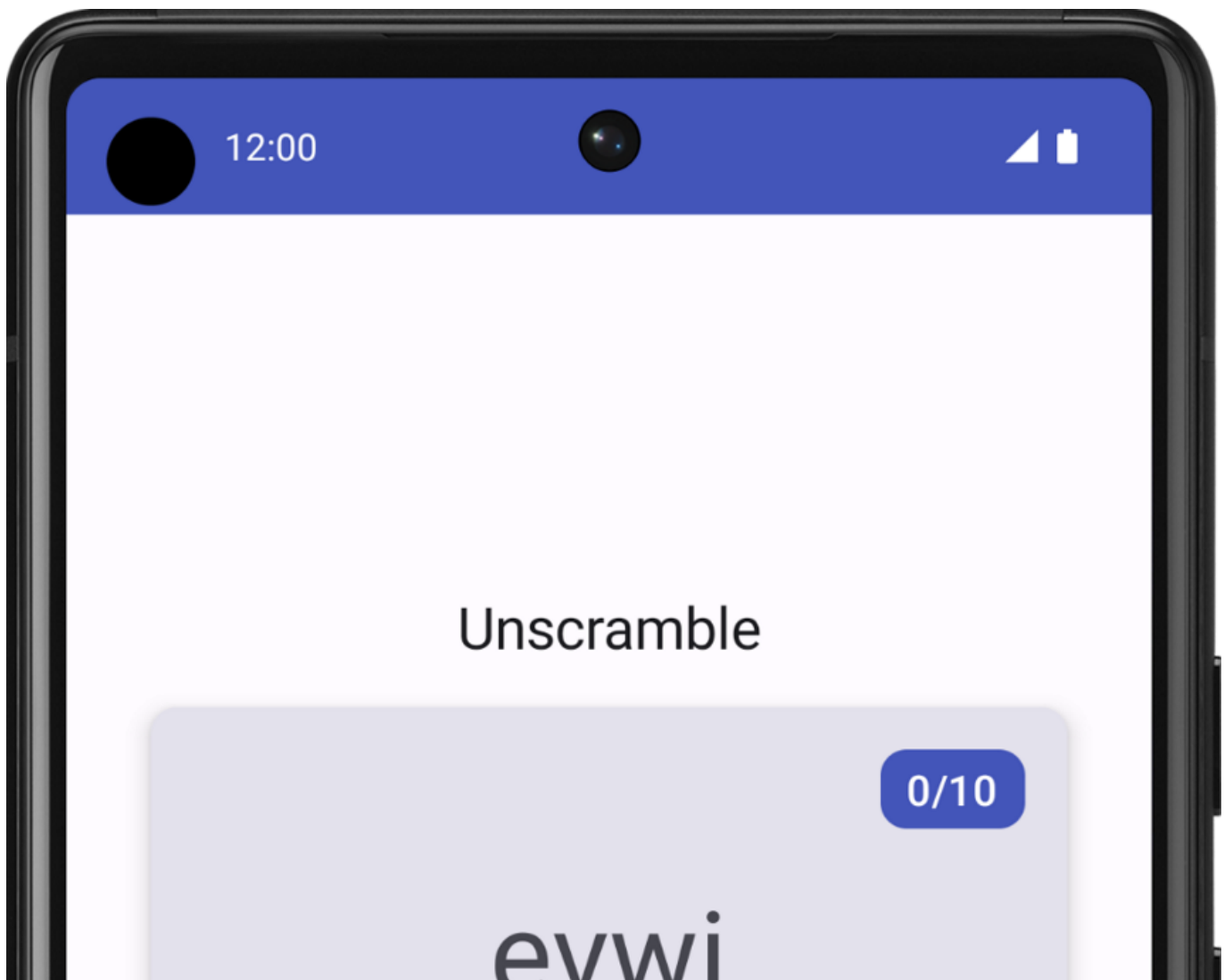
```
fun GameLayout(  
    currentScrambledWord: String,  
    userGuess: String,  
    onUserGuessChanged: (String) -> Unit,  
    onKeyboardDone: () -> Unit,  
    modifier: Modifier = Modifier  
) {  
    Column(  
        verticalArrangement = Arrangement.spacedBy(24.dp)  
    ) {
```

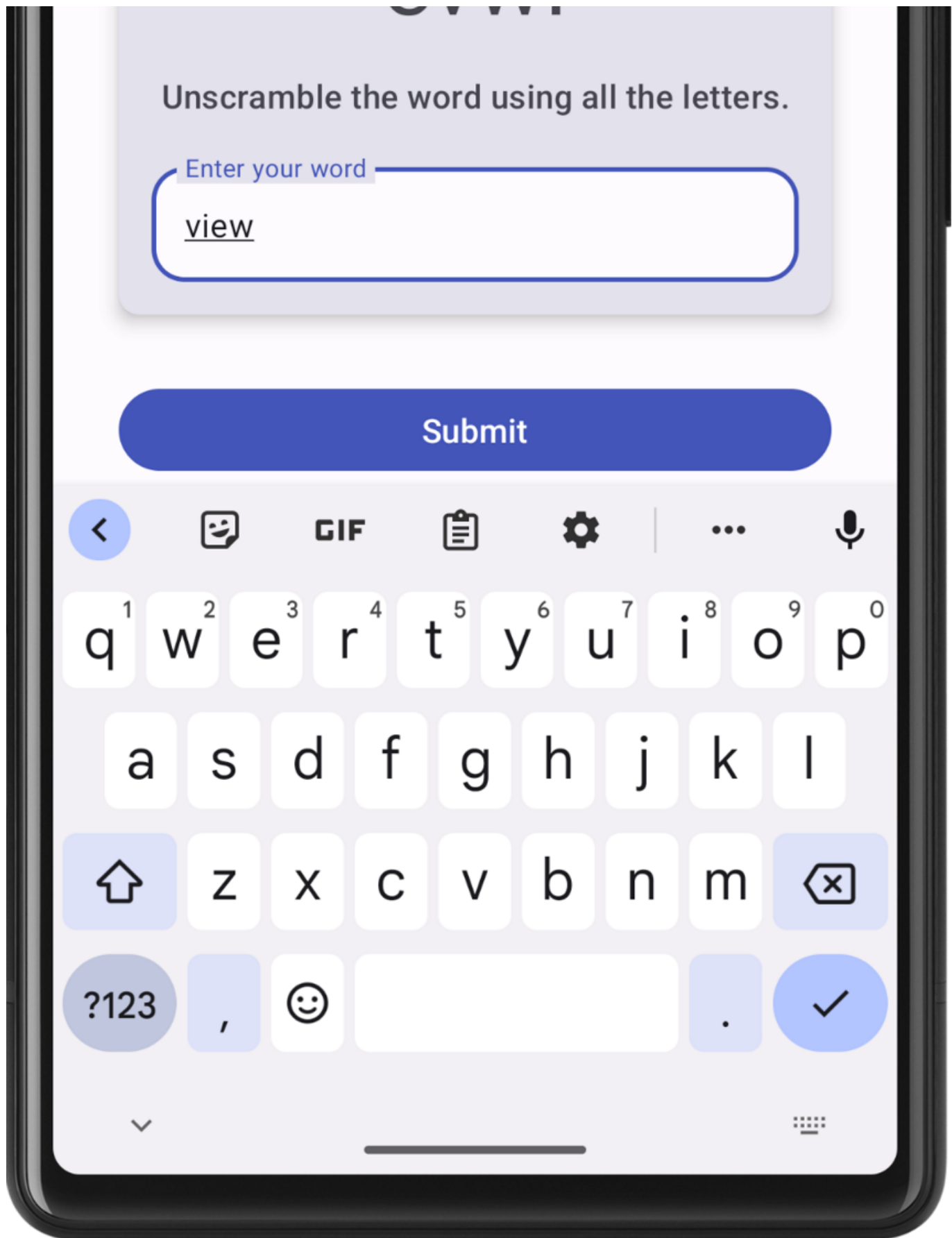
```
        //...
        OutlinedTextField(
            value = userGuess,
            //..
        )
    }
}
```

- В функции `GameScreen` обновите вызов функции `GameLayout()`, чтобы включить параметр `userGuess`.

```
GameLayout(
    currentScrambledWord = gameUiState.currentScrambledWord,
    userGuess = gameViewModel.userGuess,
    onUserGuessChanged = { gameViewModel.updateUserGuess(it) },
    onKeyboardDone = { },
    //...
)
```

- Создайте и запустите свое приложение. Попробуйте угадать и ввести слово. Текстовое поле может отображать угаданное пользователем слово.





{style="width:500px"}

### Проверка угаданного слова и обновление счета

В этом задании вы реализуете метод проверки слова, которое угадывает пользователь, а затем либо обновите счет игры, либо выведете ошибку. Позже вы обновите UI состояния игры с новым счетом и

НОВЫМ СЛОВОМ.

- В GameViewModel добавьте еще один метод под названием checkUserGuess(). В функции checkUserGuess() добавьте блок if else, чтобы проверить, совпадает ли отгадка пользователя с текущим словом. Сбросьте значение userGuess в пустую строку.

```
fun checkUserGuess() {
    if (userGuess.equals(currentWord, ignoreCase = true)) {
    } else {
    }
    // Reset user guess
    updateUserGuess("")
}
```

Если пользователь угадал неправильно, установите значение isGuessedWordWrong в true.

`MutableStateFlow<T>.update()` обновляет `MutableStateFlow.value`, используя указанное значение.

```
import kotlinx.coroutines.flow.update

if (userGuess.equals(currentWord, ignoreCase = true)) {
} else {
    // User's guess is wrong, show an error
    _uiState.update { currentState ->
        currentState.copy(isGuessedWordWrong = true)
    }
}
```

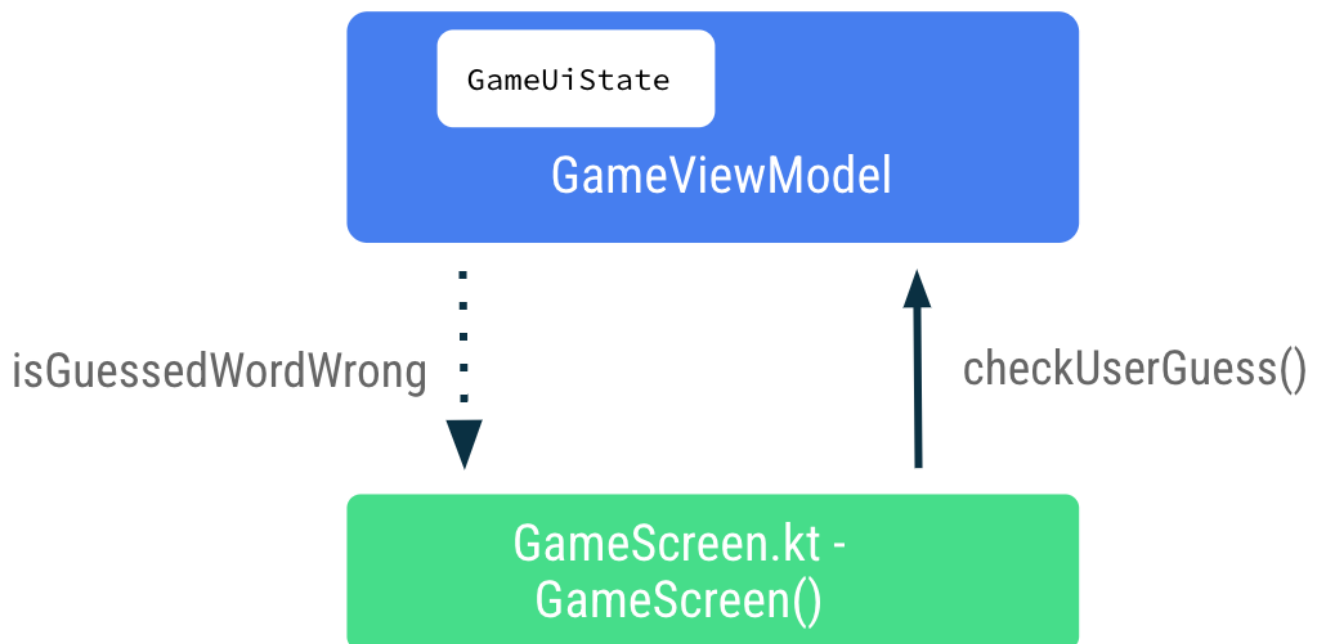
**Примечание о методе copy():** Используйте функцию `copy()` для копирования объекта, позволяя изменять некоторые его свойства, сохраняя остальные неизменными.

Пример: `val jack = User(name = «Jack», age = 1) val olderJack = jack.copy(age = 2)`

- В классе GameUiState добавьте булево значение isGuessedWordWrong и инициализируйте его значением false.

```
data class GameUiState(
    val currentScrambledWord: String = "",
    val isGuessedWordWrong: Boolean = false,
)
```

- Далее вы передаете обратный вызов события checkUserGuess() вверх от GameScreen к ViewModel, когда пользователь нажимает кнопку Submit или клавишу done на клавиатуре. Передайте данные gameUiState.isGuessedWordWrong вниз из ViewModel в GameScreen, чтобы установить ошибку в текстовом поле.



- В файле GameScreen.kt в конце композитной функции GameScreen() вызовите gameViewModel.checkUserGuess() внутри лямбда-выражения onClick кнопки Submit.

```

Button(
    modifier = modifier
        .fillMaxWidth()
        .weight(1f)
        .padding(start = 8.dp),
    onClick = { gameViewModel.checkUserGuess() }
) {
    Text(stringResource(R.string.submit))
}
    
```

- В составной функции GameScreen() обновите вызов функции GameLayout(), чтобы передать gameViewModel.checkUserGuess() в лямбда-выражении onKeyboardDone.

```

GameLayout(
    currentScrambledWord = gameUiState.currentScrambledWord,
    userGuess = gameViewModel.userGuess,
    onUserGuessChanged = { gameViewModel.updateUserGuess(it) },
    onKeyboardDone = { gameViewModel.checkUserGuess() }
)
    
```

- В составной функции GameLayout() добавьте параметр функции для булевого значения isGuessWrong. Установите параметр isError поля OutlinedTextField в значение isGuessWrong, чтобы отобразить ошибку в текстовом поле, если пользователь угадал неправильно.

```

fun GameLayout(
    currentScrambledWord: String,
    isGuessWrong: Boolean,
    userGuess: String,
    onUserGuessChanged: (String) -> Unit,
    onKeyboardDone: () -> Unit,
    modifier: Modifier = Modifier
) {
    Column(
        // ,...
        OutlinedTextField(
            // ...
            isError = isGuessWrong,
            keyboardOptions = KeyboardOptions.Default.copy(
                imeAction = ImeAction.Done
            ),
            keyboardActions = KeyboardActions(
                onDone = { onKeyboardDone() }
            ),
        )
    )
}

```

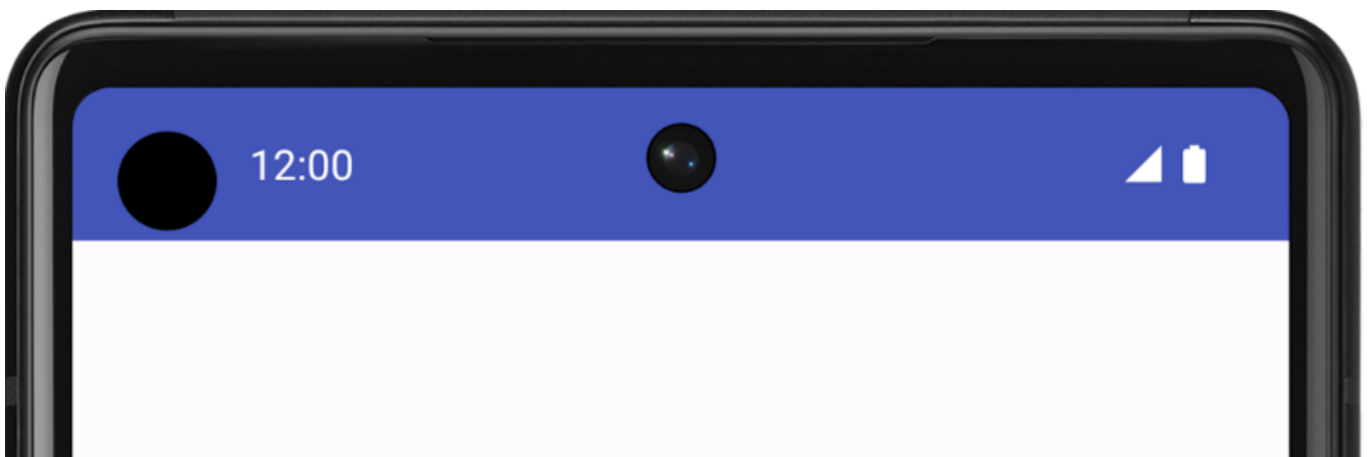
- В композитной функции GameScreen() обновите вызов функции GameLayout(), чтобы передать isGuessWrong.

```

GameLayout(
    currentScrambledWord = gameUiState.currentScrambledWord,
    userGuess = gameViewModel.userGuess,
    onUserGuessChanged = { gameViewModel.updateUserGuess(it) },
    onKeyboardDone = { gameViewModel.checkUserGuess() },
    isGuessWrong = gameUiState.isGuessedWordWrong,
    // ...
)

```

- Создайте и запустите свое приложение. Введите неверное предположение и нажмите кнопку Отправить. Обратите внимание, что текстовое поле стало красным, указывая на ошибку.



# Unscramble

0/10

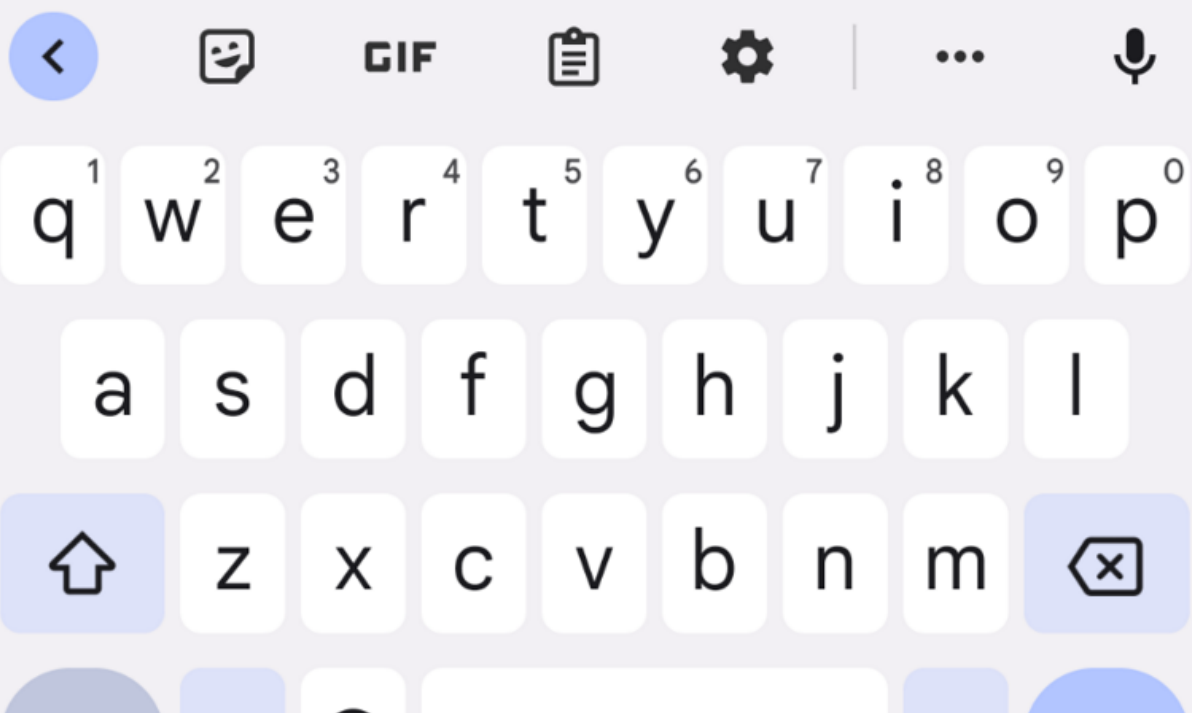
fere

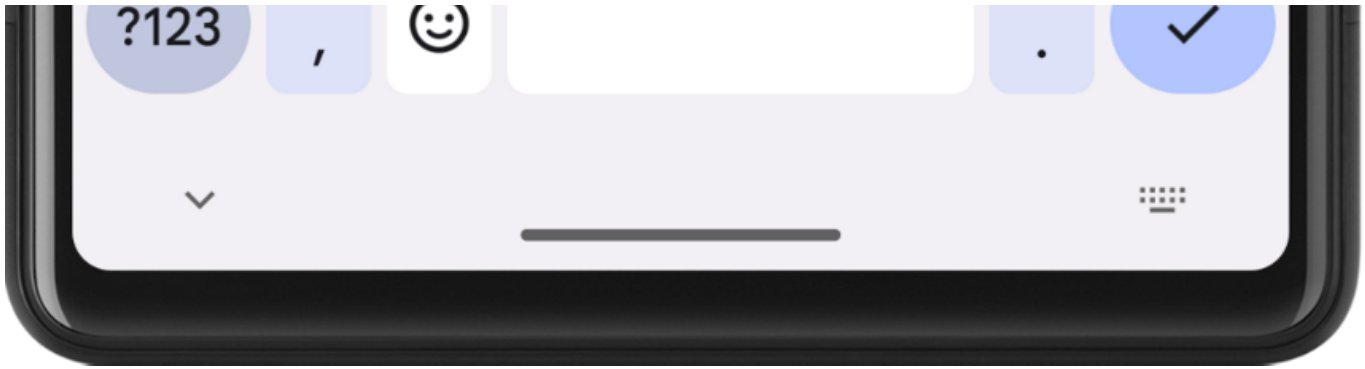
Unscramble the word using all the letters.

Enter your word

|

Submit





```
{style="width:500px;"}
```

Обратите внимание, что в текстовом поле по-прежнему написано «Введите слово». Чтобы сделать его удобным для пользователя, нужно добавить текст ошибки, указывающий на то, что слово введено неправильно.

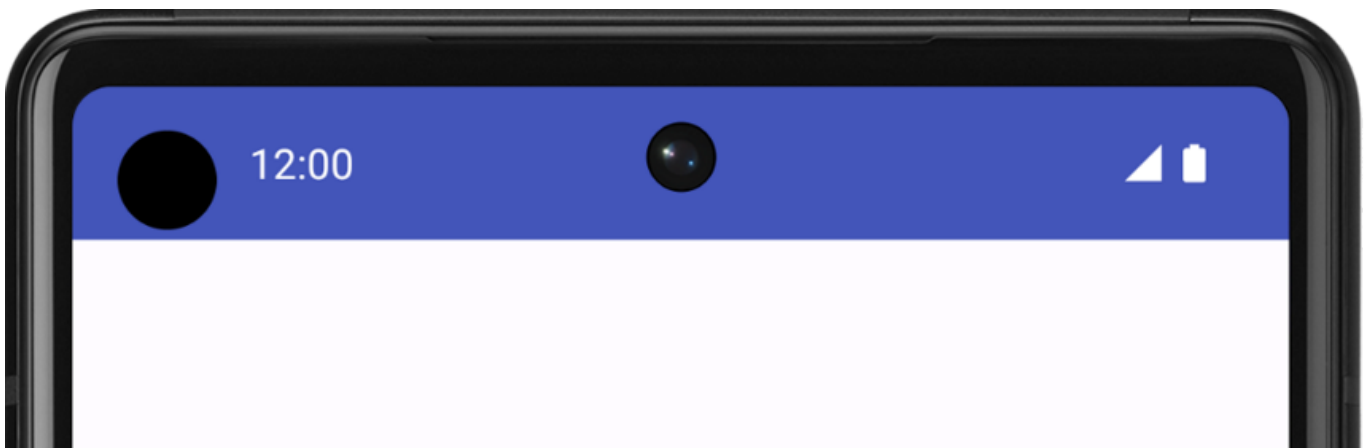
- В файле `GameScreen.kt`, в композите `GameLayout()`, обновите параметр `label` текстового поля в зависимости от `isGuessWrong` следующим образом:

```
OutlinedTextField(
    // ...
    label = {
        if (isGuessWrong) {
            Text(stringResource(R.string.wrong_guess))
        } else {
            Text(stringResource(R.string.enter_your_word))
        }
    },
    // ...
)
```

- В файле `strings.xml` добавьте строку для метки ошибки.

```
<string name="wrong_guess">Wrong Guess!</string>
```

- Создайте и запустите свое приложение снова. Введите неверное предположение и нажмите кнопку Отправить. Обратите внимание на надпись об ошибке.



# Unscramble

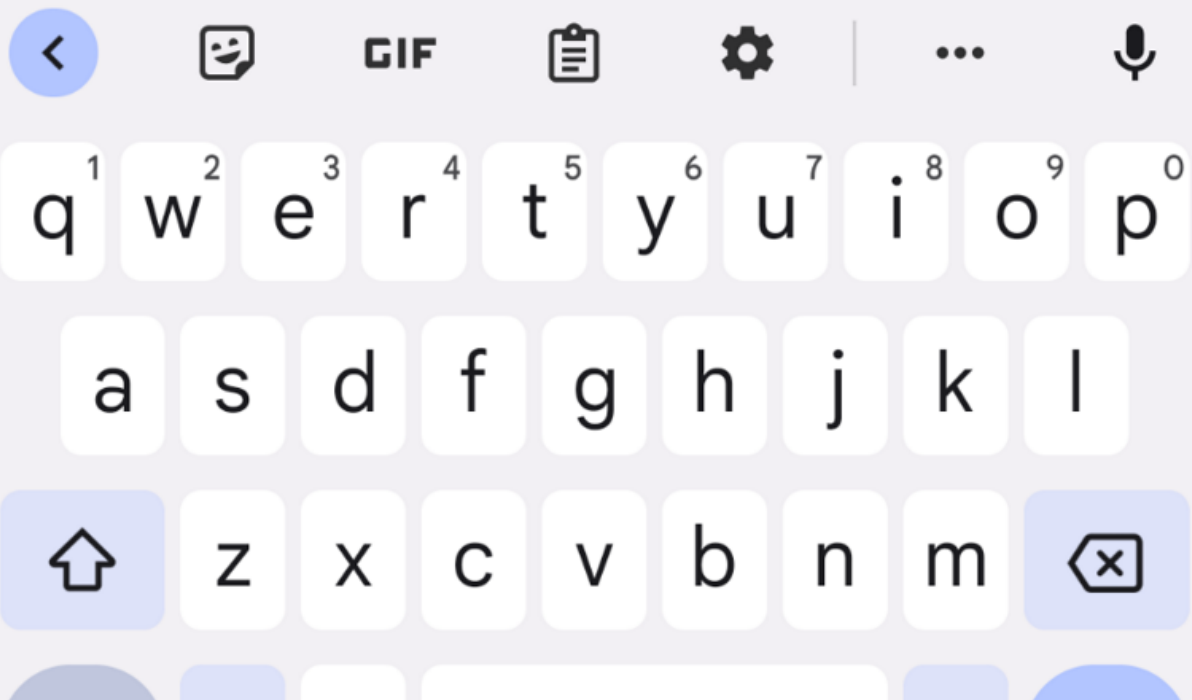
0/10

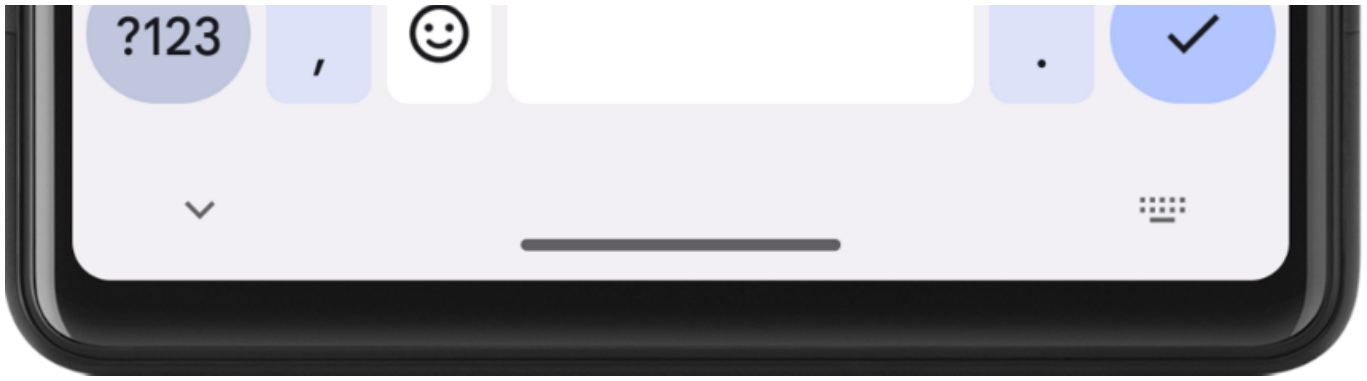
viewi

Unscramble the word using all the letters.

Wrong Guess!

Submit





{style="width:500px"}

## Обновление счета и количества слов

В этой задаче вы обновляете счет и количество слов по мере того, как пользователь играет в игру. Счет должен быть частью `_uiState`.

- В `GameUiState` добавьте переменную `score` и инициализируйте ее нулем.

```
data class GameUiState(
    val currentScrambledWord: String = "",
    val isGuessedWordWrong: Boolean = false,
    val score: Int = 0
)
```

- Чтобы обновить значение очков, в `GameViewModel`, в функции `checkUserGuess()`, внутри условия `if` для случая, когда пользователь угадал правильно, увеличьте значение очков.

```
import com.example.unscramble.data.SCORE_INCREASE

fun checkUserGuess() {
    if (userGuess.equals(currentWord, ignoreCase = true)) {
        // User's guess is correct, increase the score
        val updatedScore = _uiState.value.score.plus(SCORE_INCREASE)
    } else {
        //...
    }
}
```

- В `GameViewModel` добавьте еще один метод под названием `updateGameState`, который будет обновлять счет, увеличивать текущее количество слов и выбирать новое слово из файла `WordsData.kt`. В качестве параметра добавьте `Int` с именем `updatedScore`. Обновите переменные пользовательского интерфейса состояния игры следующим образом:

```
private fun updateGameState(updatedScore: Int) {
    _uiState.update { currentState ->
        currentState.copy(
```

```

        isGuessedWordWrong = false,
        currentScrambledWord = pickRandomWordAndShuffle(),
        score = updatedScore
    )
}
}

```

- В функции `checkUserGuess()`, если пользователь угадал правильно, выполните вызов `updateGameState` с обновленным счетом, чтобы подготовить игру к следующему раунду.

```

fun checkUserGuess() {
    if (userGuess.equals(currentWord, ignoreCase = true)) {
        // User's guess is correct, increase the score
        // and call updateGameState() to prepare the game for next round
        val updatedScore = _uiState.value.score.plus(SCORE_INCREASE)
        updateGameState(updatedScore)
    } else {
        //...
    }
}

```

- Завершенная функция `checkUserGuess()` должна выглядеть следующим образом:

```

fun checkUserGuess() {
    if (userGuess.equals(currentWord, ignoreCase = true)) {
        // User's guess is correct, increase the score
        // and call updateGameState() to prepare the game for next round
        val updatedScore = _uiState.value.score.plus(SCORE_INCREASE)
        updateGameState(updatedScore)
    } else {
        // User's guess is wrong, show an error
        _uiState.update { currentState ->
            currentState.copy(isGuessedWordWrong = true)
        }
    }
    // Reset user guess
    updateUserGuess("")
}

```

Далее, аналогично обновлению счета, вам нужно обновить количество слов.

- Добавьте еще одну переменную для подсчета в `GameUiState`. Назовите ее `currentWordCount` и инициализируйте ее значением 1.

```

data class GameUiState(
    val currentScrambledWord: String = "",
    val currentWordCount: Int = 1,

```

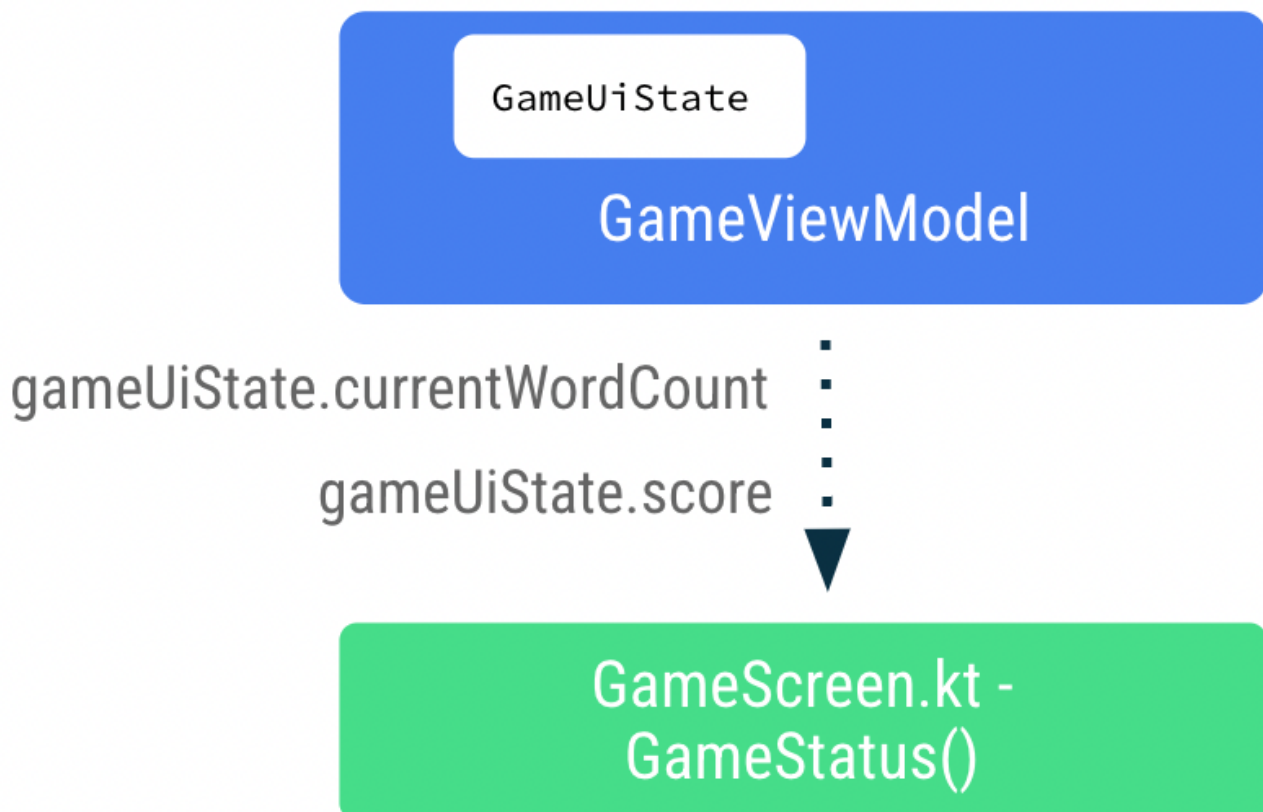
```
val score: Int = 0,
val isGuessedWordWrong: Boolean = false,
)
```

- В файле GameViewModel.kt в функции updateGameState() увеличьте количество слов, как показано ниже. Функция updateGameState() вызывается для подготовки игры к следующему раунду.

```
private fun updateGameState(updatedScore: Int) {
    _uiState.update { currentState ->
        currentState.copy(
            //...
            currentWordCount = currentState.currentWordCount.inc(),
        )
    }
}
```

## Передача результатов и количества слов

- Выполните следующие шаги, чтобы передать данные о счете и количестве слов из ViewModel в GameScreen.



- В файле GameScreen.kt в композитной функции GameLayout() добавьте в качестве аргумента word count и передайте аргументы формата wordCount элементу text.

```

fun GameLayout(
    onUserGuessChanged: (String) -> Unit,
    onKeyboardDone: () -> Unit,
    wordCount: Int,
    //...
) {
    //...

    Card(
        //...
    ) {
        Column(
            // ...
        ) {
            Text(
                //..
                text = stringResource(R.string.word_count, wordCount),
                style = typography.titleMedium,
                color = colorScheme.onPrimary
            )

            // ...
        }
    }
}

```

- Обновите вызов функции GameLayout(), чтобы включить подсчет количества слов.

```

GameLayout(
    userGuess = gameViewModel.userGuess,
    wordCount = gameUiState.currentWordCount,
    //...
)

```

- В составной функции GameScreen() обновите вызов функции GameStatus(), чтобы включить параметры счета. Передайте счет из gameUiState.

```

GameStatus(score = gameUiState.score, modifier = Modifier.padding(20.dp))

```

- Создайте и запустите приложение.
- Введите угаданное слово и нажмите кнопку **Отправить**. Заметьте, что счет и количество слов обновляются.
- Нажмите кнопку **Пропустить** и заметите, что ничего не происходит.

Чтобы реализовать функцию пропуска, вам нужно передать обратный вызов события пропуска в модель GameViewModel.

- В файле `GameScreen.kt` в составной функции `GameScreen()` сделайте вызов `gameViewModel.skipWord()` в лямбда-выражении `onClick`.

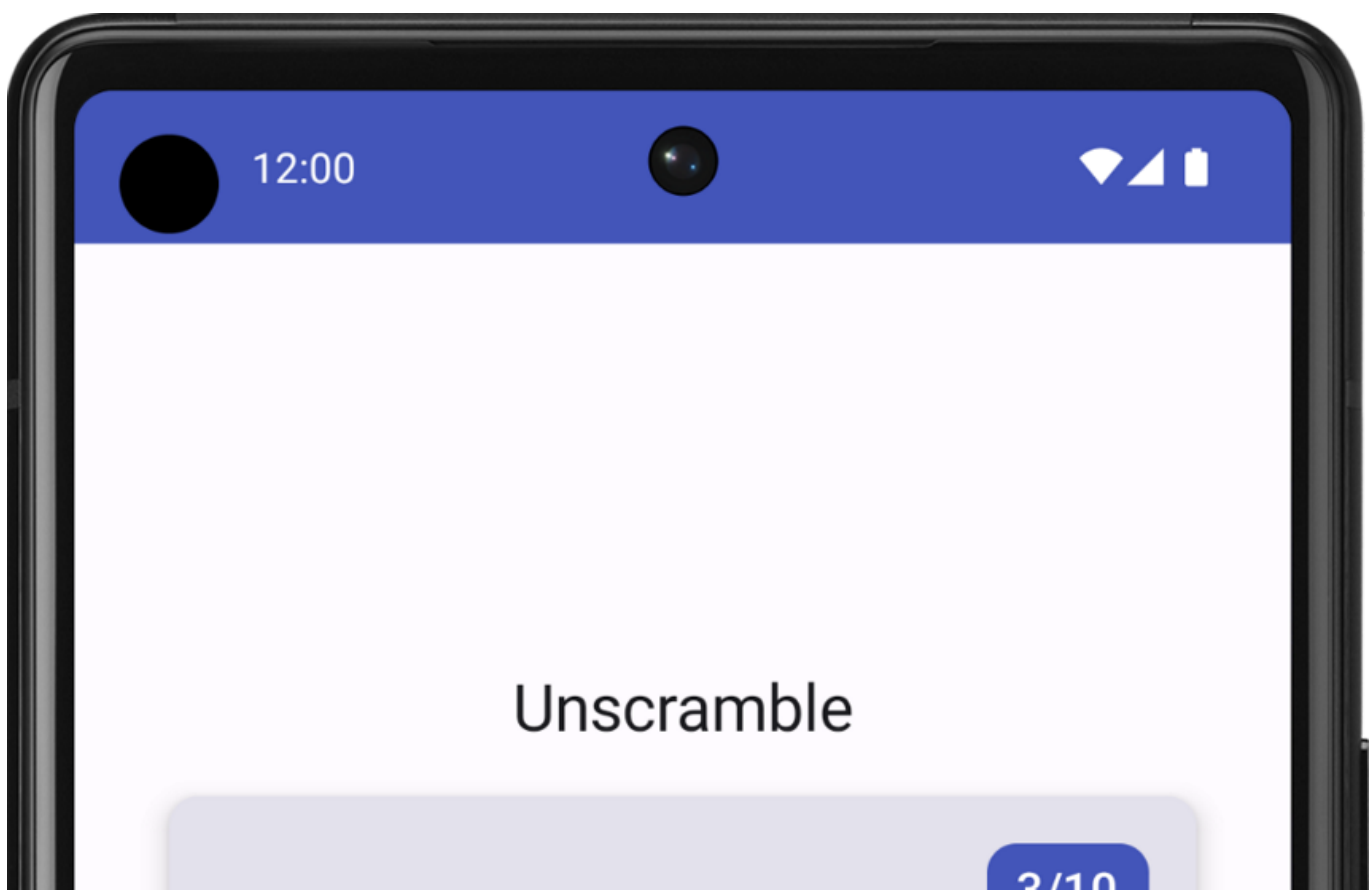
Android Studio выдает ошибку, поскольку функция еще не реализована. Вы исправляете эту ошибку в следующем шаге, добавляя метод `skipWord()`. Когда пользователь пропускает слово, вам нужно обновить игровые переменные и подготовить игру к следующему раунду.

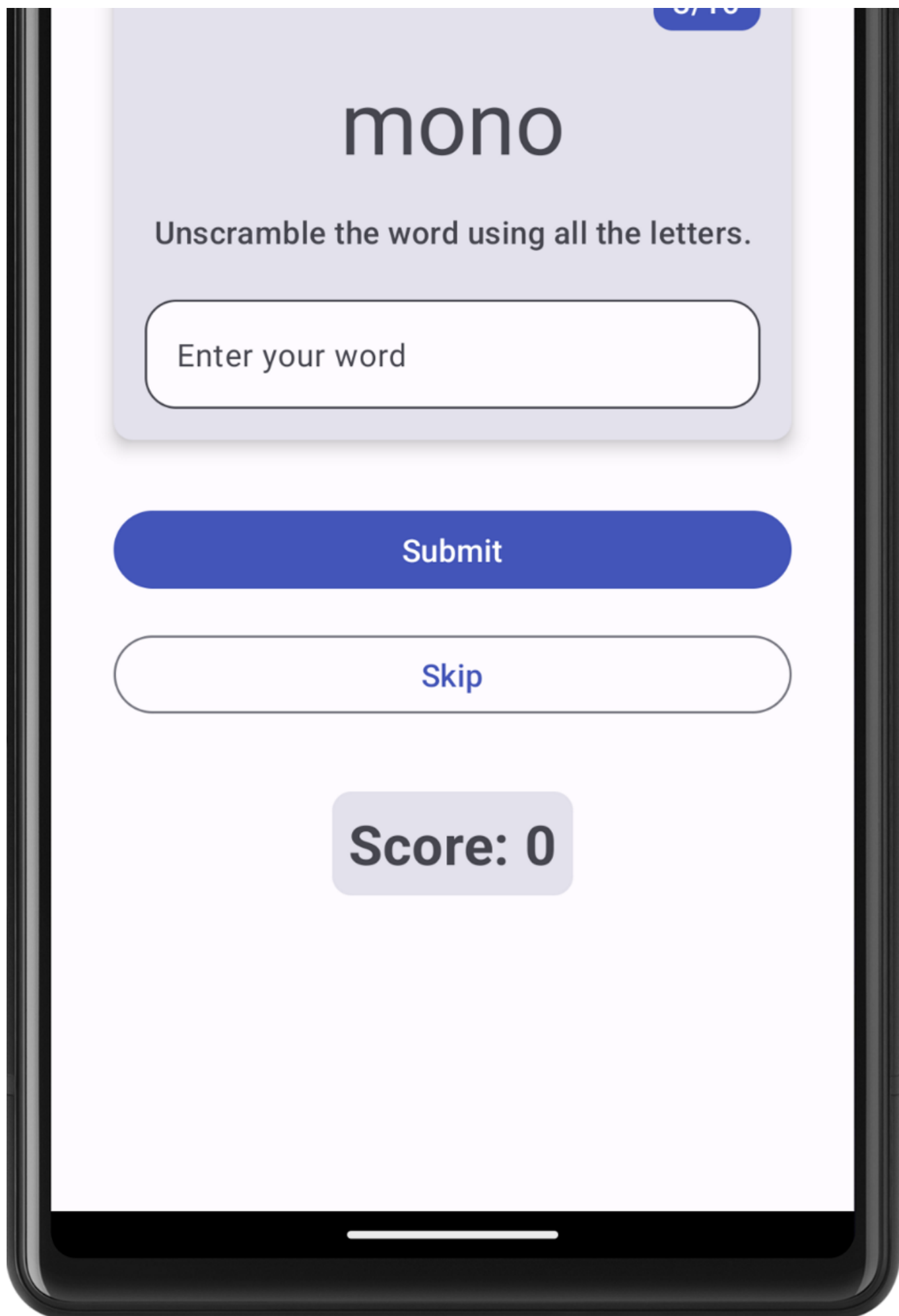
```
OutlinedButton(  
    onClick = { gameViewModel.skipWord() },  
    modifier = Modifier.fillMaxWidth()  
) {  
    //...  
}
```

- В `GameViewModel` добавьте метод `skipWord()`. Внутри функции `skipWord()` сделайте вызов `updateGameState()`, передав счет и сбросив пользовательскую догадку.

```
fun skipWord() {  
    updateGameState(_uiState.value.score)  
    // Reset user guess  
    updateUserGuess("")  
}
```

- Запустите приложение и сыграйте в игру. Теперь у вас должна появиться возможность пропускать слова.



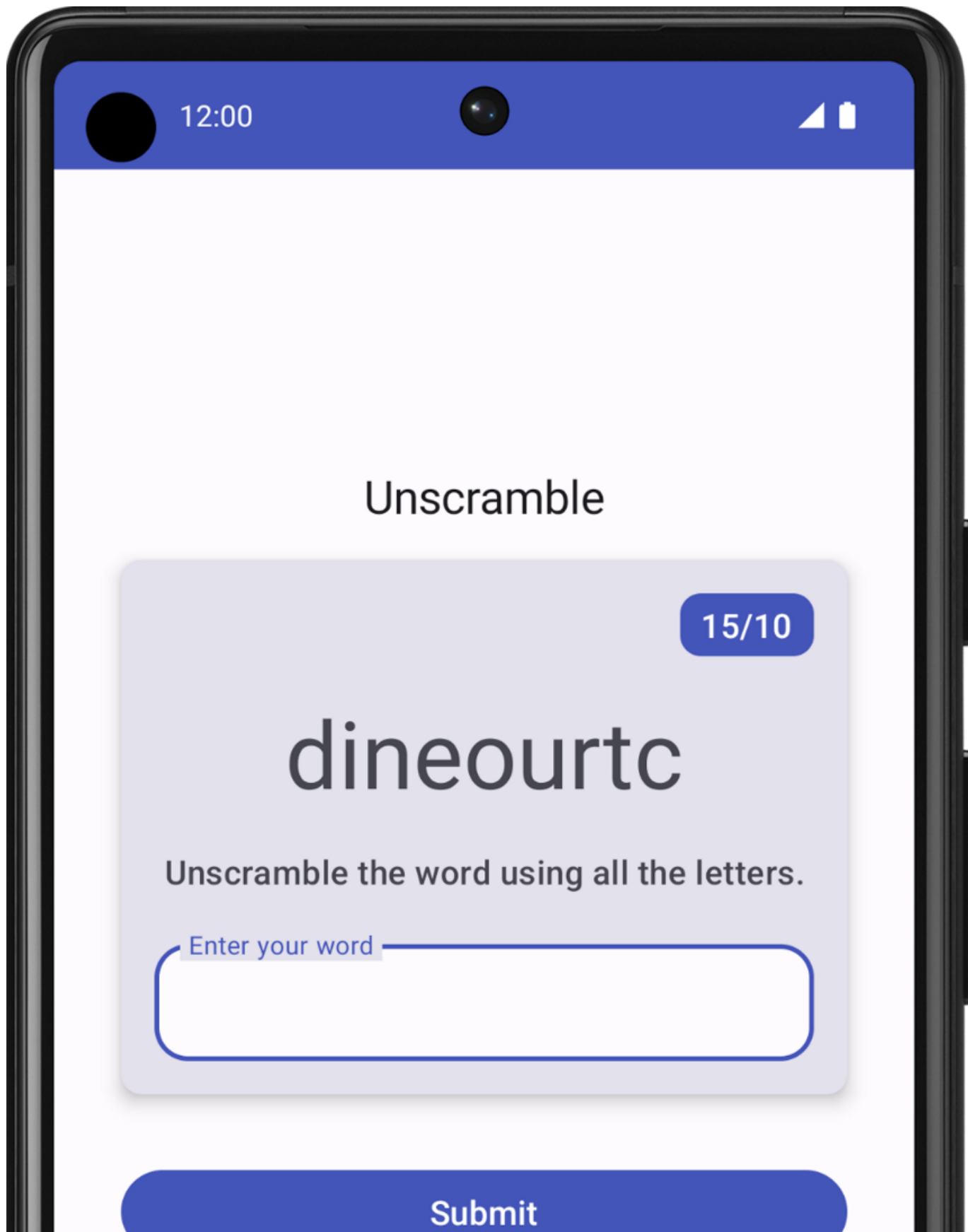


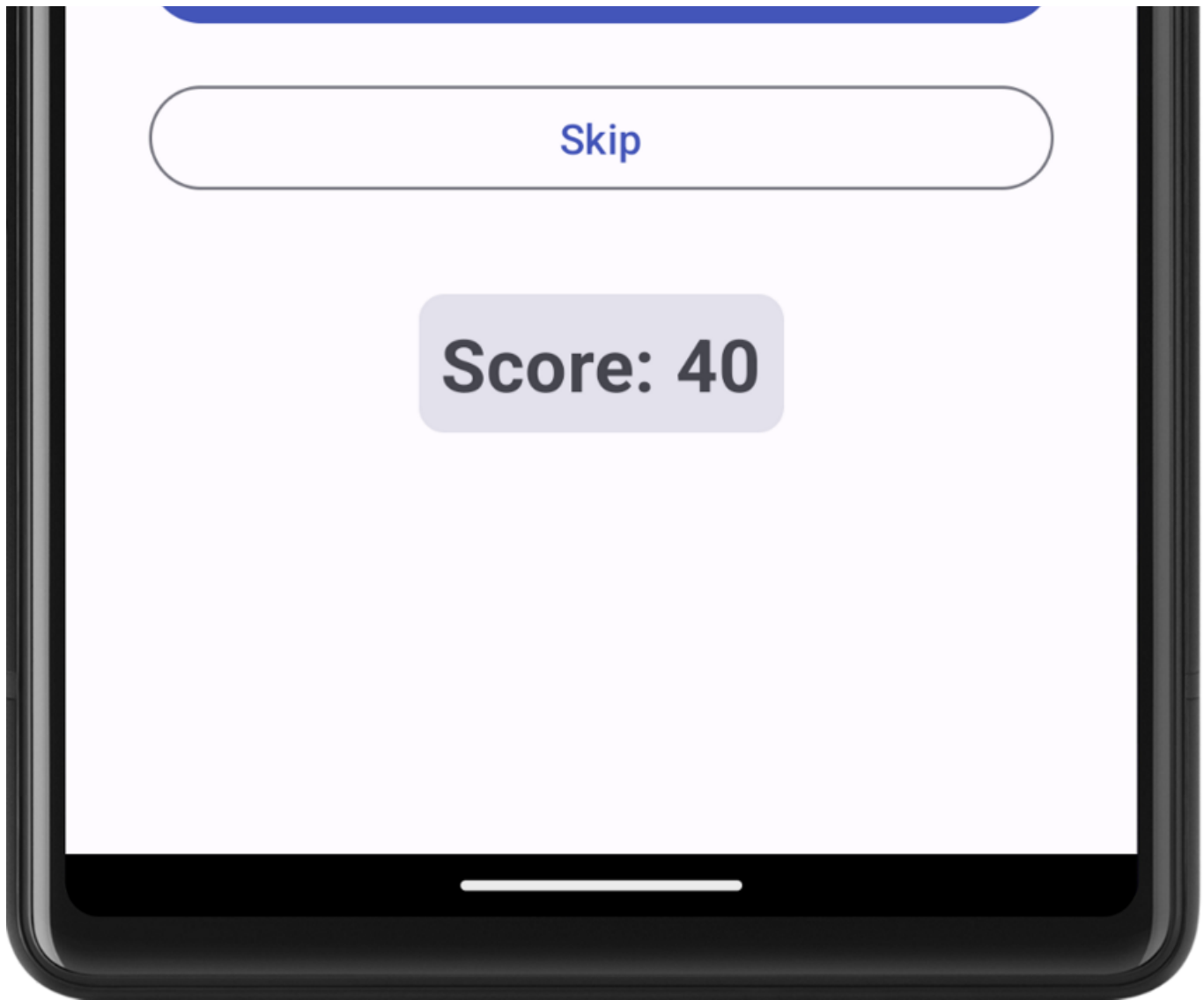
{style="width:500px"}

Вы можете играть в игру и после 10 слов. В следующем задании вы справитесь с последним раундом игры.

### Обработка последнего раунда игры

В текущей реализации пользователи могут пропускать или играть дальше 10 слов. В этой задаче вы добавляете логику для завершения игры.





```
{style="width:500px"}
```

Чтобы реализовать логику завершения игры, сначала нужно проверить, достиг ли пользователь максимального количества слов.

- В GameViewModel добавьте блок if-else и переместите существующее тело функции внутрь блока else.
- Добавьте условие if для проверки того, что размер usedWords равен MAX\_NO\_OF\_WORDS.

```
import com.example.android.unscramble.data.MAX_NO_OF_WORDS

private fun updateGameState(updatedScore: Int) {
    if (usedWords.size == MAX_NO_OF_WORDS){
        //Last round in the game
    } else{
        // Normal round in the game
        _uiState.update { currentState ->
            currentState.copy(
                isGuessedWordWrong = false,
                currentScrambledWord = pickRandomWordAndShuffle(),
                currentWordCount = currentState.currentWordCount.inc(),
            )
        }
    }
}
```

```

        score = updatedScore
    )
}
}
}

```

- Внутри блока if добавьте булевский флаг isGameOver и установите его в true, чтобы указать на окончание игры.
- Обновите счет и сбросьте isGuessedWordWrong внутри блока if. Следующий код показывает, как должна выглядеть ваша функция:

```

private fun updateGameState(updatedScore: Int) {
    if (usedWords.size == MAX_NO_OF_WORDS){
        //Last round in the game, update isGameOver to true, don't pick a new word
        _uiState.update { currentState ->
            currentState.copy(
                isGuessedWordWrong = false,
                score = updatedScore,
                isGameOver = true
            )
        }
    } else{
        // Normal round in the game
        _uiState.update { currentState ->
            currentState.copy(
                isGuessedWordWrong = false,
                currentScrambledWord = pickRandomWordAndShuffle(),
                currentWordCount = currentState.currentWordCount.inc(),
                score = updatedScore
            )
        }
    }
}

```

- В GameUiState добавьте булеву переменную isGameOver и установите ее в значение false.

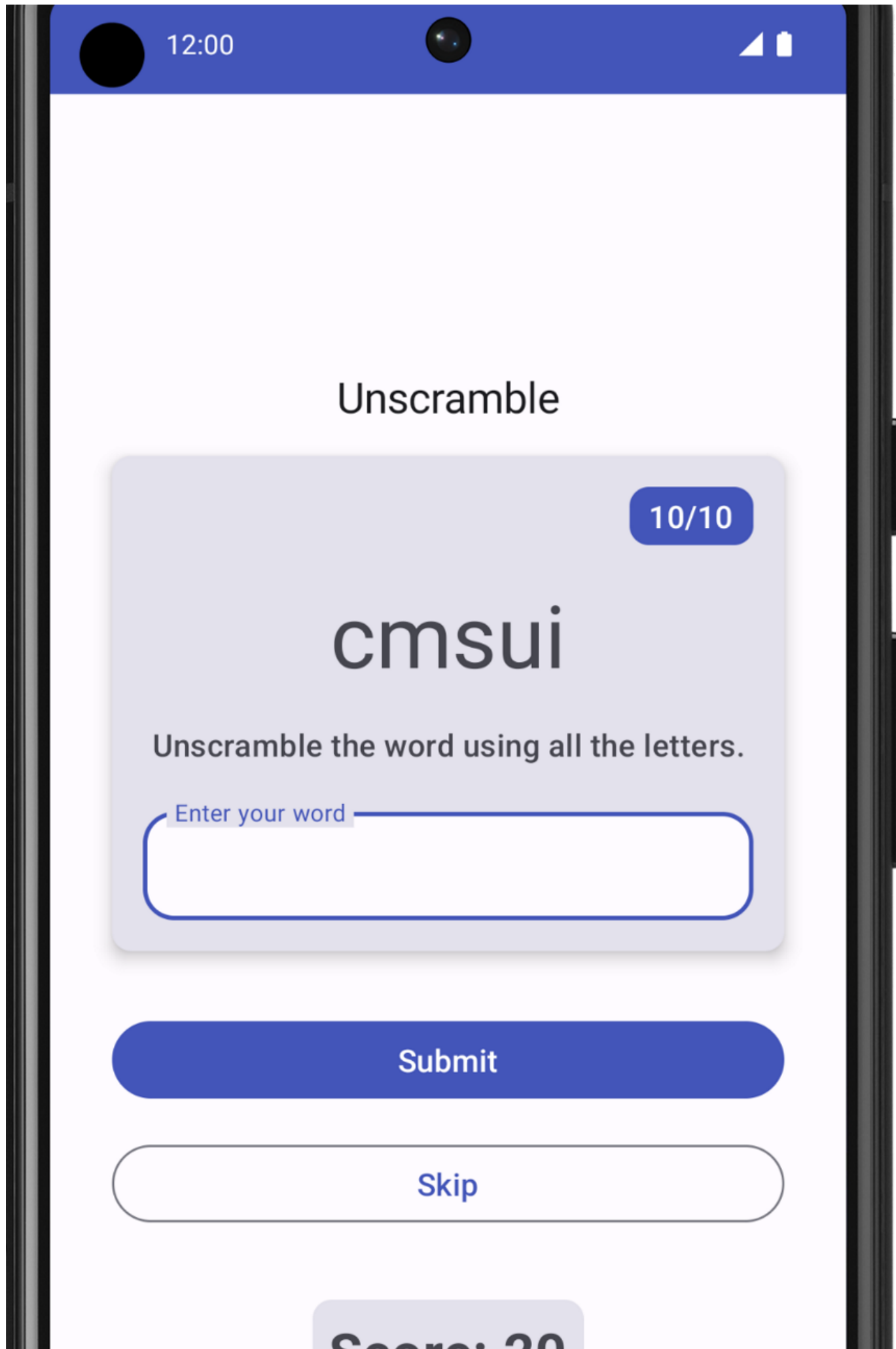
```

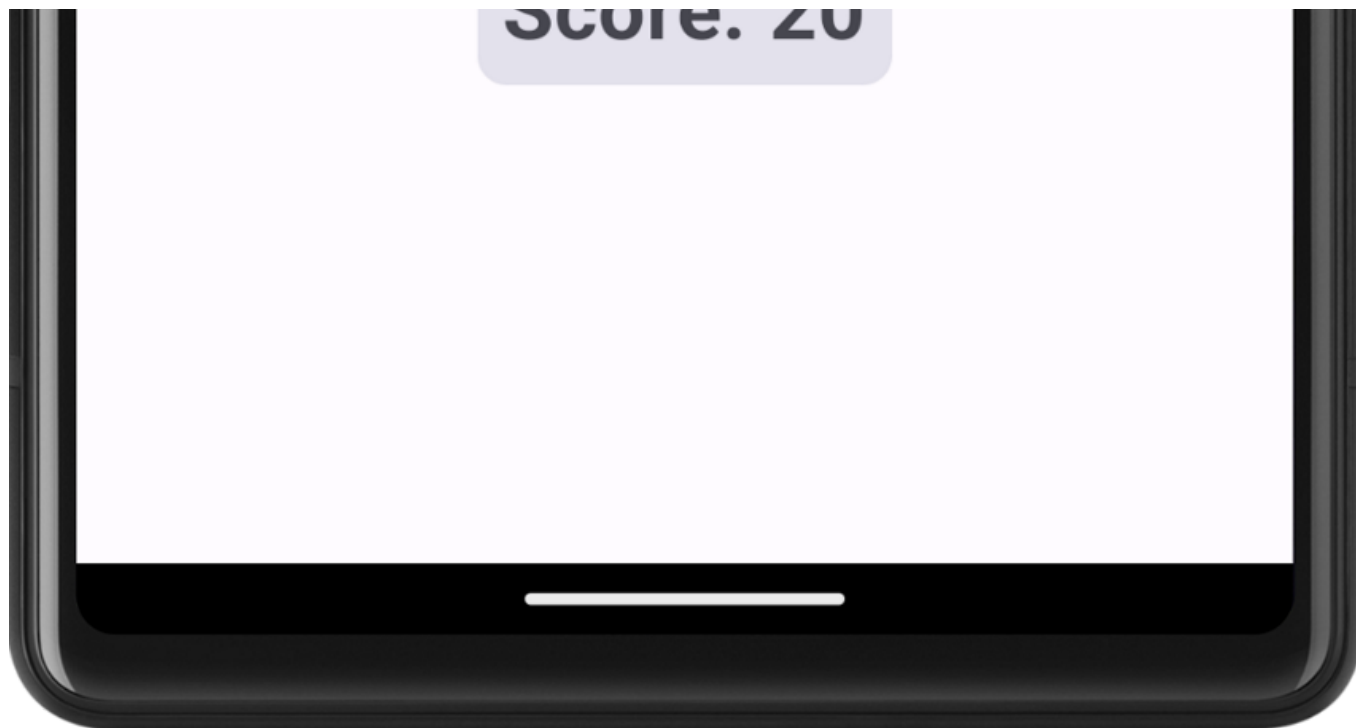
data class GameUiState(
    val currentScrambledWord: String = "",
    val currentWordCount: Int = 1,
    val score: Int = 0,
    val isGuessedWordWrong: Boolean = false,
    val isGameOver: Boolean = false
)

```

- Запустите приложение и начните игру. Вы не можете играть дальше 10 слов.







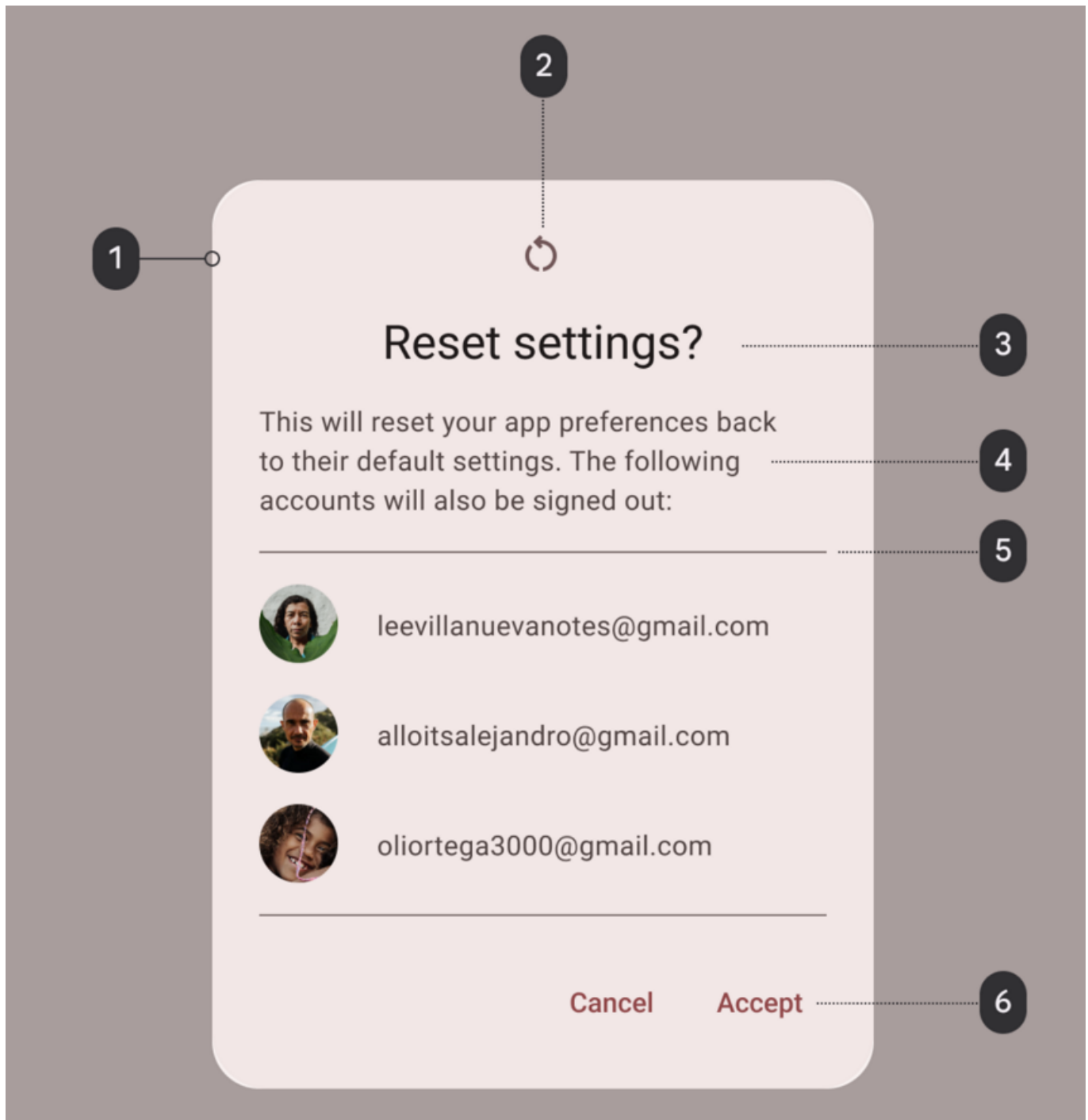
Когда игра заканчивается, было бы неплохо сообщить об этом пользователю и спросить, не хочет ли он сыграть еще раз. Вы реализуете эту возможность в следующем задании.

### Отображение диалога об окончании игры

В этой задаче вы передаете данные `isGameOver` в `GameScreen` из `ViewModel` и используете их для отображения диалогового окна с опциями завершения или перезапуска игры.

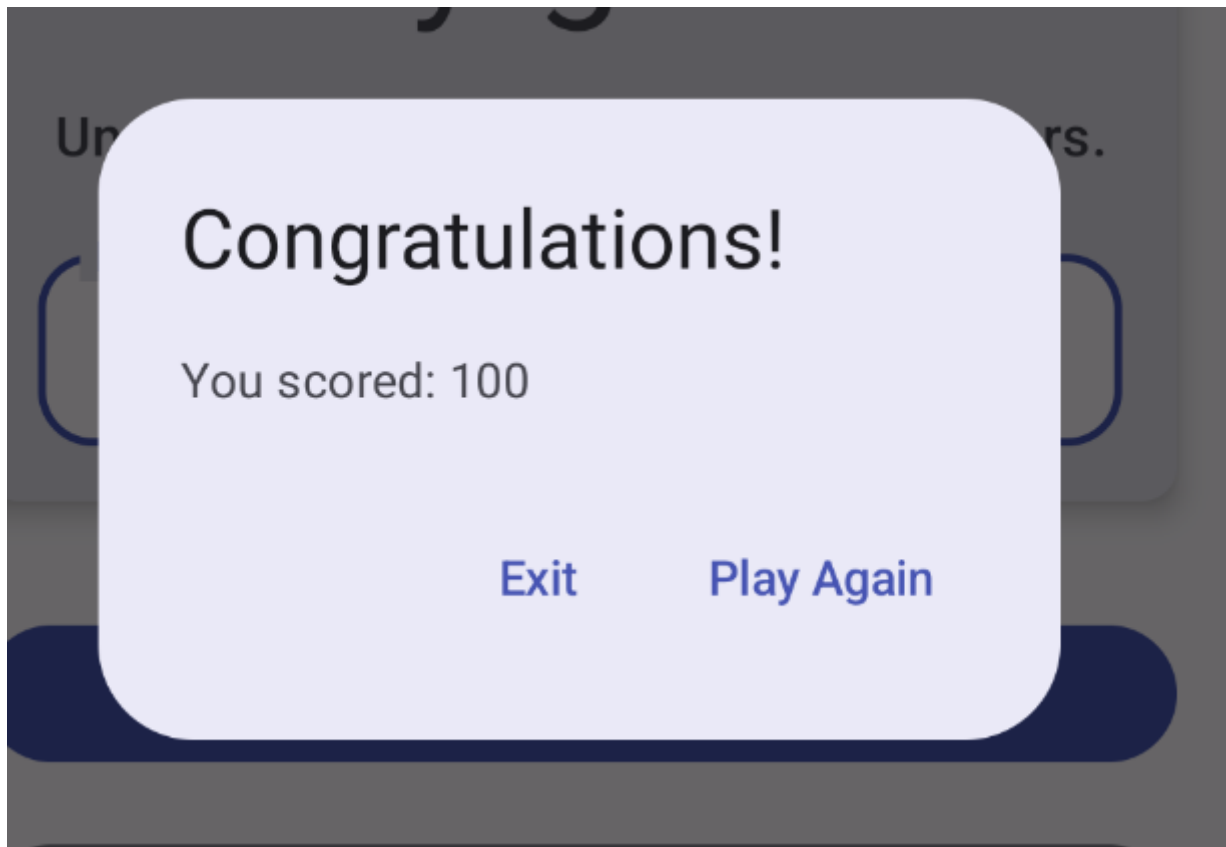
**Диалог** - это небольшое окно, в котором пользователю предлагается принять решение или ввести дополнительную информацию. Обычно диалог не занимает весь экран и требует от пользователя совершить какое-то действие, прежде чем он сможет продолжить. В Android предусмотрены различные типы диалоговых окон. В этом уроке вы узнаете о диалоговых окнах оповещения.

### Анатомия диалогового окна оповещения



- Контейнер
- Иконка (необязательно)
- Заголовок (необязательно)
- Сопровождающий текст
- Разделитель (необязательно)
- Действия

Файл `GameScreen.kt` в стартовом коде уже содержит функцию, которая отображает диалог предупреждения с опциями выхода или перезапуска игры.



```
@Composable
private fun FinalScoreDialog(
    onPlayAgain: () -> Unit,
    modifier: Modifier = Modifier
) {
    val activity = (LocalContext.current as Activity)

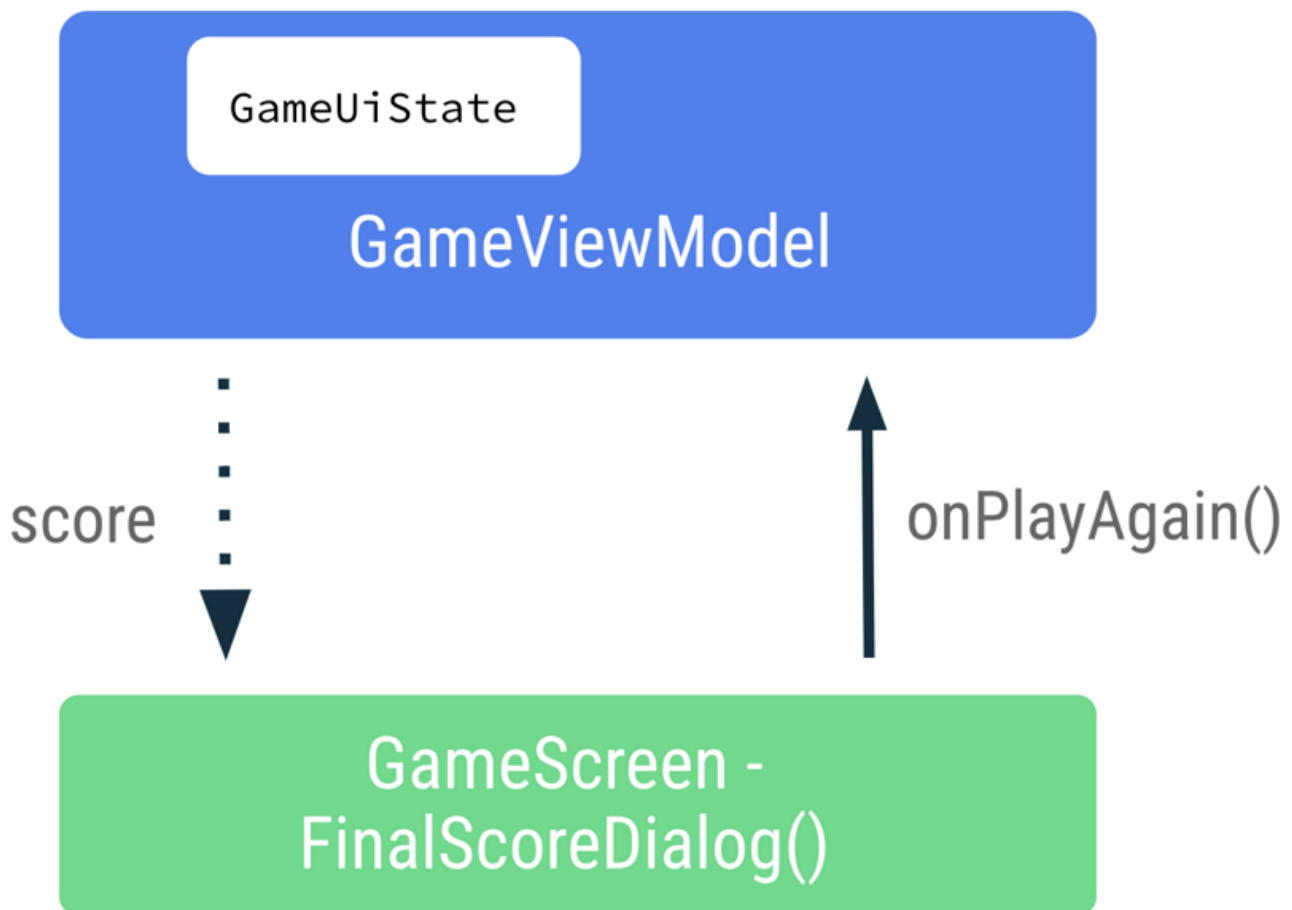
    AlertDialog(
        onDismissRequest = {
            // Dismiss the dialog when the user clicks outside the dialog or on the
            back
            // button. If you want to disable that functionality, simply use an
            empty
            // onDismissRequest.
        },
        title = { Text(stringResource(R.string.congratulations)) },
        text = { Text(stringResource(R.string.you_scored, 0)) },
        modifier = modifier,
        dismissButton = {
            TextButton(
                onClick = {
                    activity.finish()
                }
            ) {
                Text(text = stringResource(R.string.exit))
            }
        },
        confirmButton = {
            TextButton(
                onClick = {
                    onPlayAgain()
                }
            ) {
                Text(text = stringResource(R.string.play_again))
            }
        }
    )
}
```

```

        onClick = {
            onPlayAgain()
        }
    ) {
        Text(text = stringResource(R.string.play_again))
    }
}
)
}

```

В этой функции параметры `title` и `text` отображают заголовок и сопроводительный текст в диалоге оповещения. Кнопки `dismissButton` и `confirmButton` являются текстовыми кнопками. В параметре `dismissButton` отображается текст `Exit` и приложение завершает свою деятельность. В параметре `confirmButton` вы перезапускаете игру и отображаете текст `Play Again`.



В файле `GameScreen.kt` в функции `FinalScoreDialog()` обратите внимание на параметр `score` для отображения счета игры в диалоге оповещения.

```

@Composable
private fun FinalScoreDialog(
    score: Int,
    onPlayAgain: () -> Unit,

```

```
modifier: Modifier = Modifier
) {
```

В функции `FinalScoreDialog()` обратите внимание на использование лямбда-выражения параметра `text` для использования оценки в качестве аргумента формата текста диалога.

```
text = { Text(stringResource(R.string.you_scored, score)) }
```

- В файле `GameScreen.kt` в конце композитной функции `GameScreen()` после блока `Column` добавьте условие `if` для проверки `gameUiState.isGameOver`.

В блоке `if` отобразите диалог оповещения. Выполните вызов `FinalScoreDialog()`, передав в нее счет и `gameViewModel.resetGame()` для обратного вызова события `onPlayAgain`.

```
if (gameUiState.isGameOver) {
    FinalScoreDialog(
        score = gameUiState.score,
        onPlayAgain = { gameViewModel.resetGame() }
    )
}
```

Функция `resetGame()` - это обратный вызов события, которое передается от игрового экрана к `ViewModel`.

- В файле `GameViewModel.kt` вызовите функцию `resetGame()`, которая инициализирует `_uiState` и выбирает новое слово.

```
fun resetGame() {
    usedWords.clear()
    _uiState.value = GameUiState(currentScrambledWord = pickRandomWordAndShuffle())
}
```

- Создайте и запустите свое приложение. Доиграйте игру до конца и обратите внимание на диалог предупреждения с вариантами выхода из игры или повторного воспроизведения. Попробуйте варианты, отображаемые в диалоге предупреждения.







## Состояние при повороте устройства

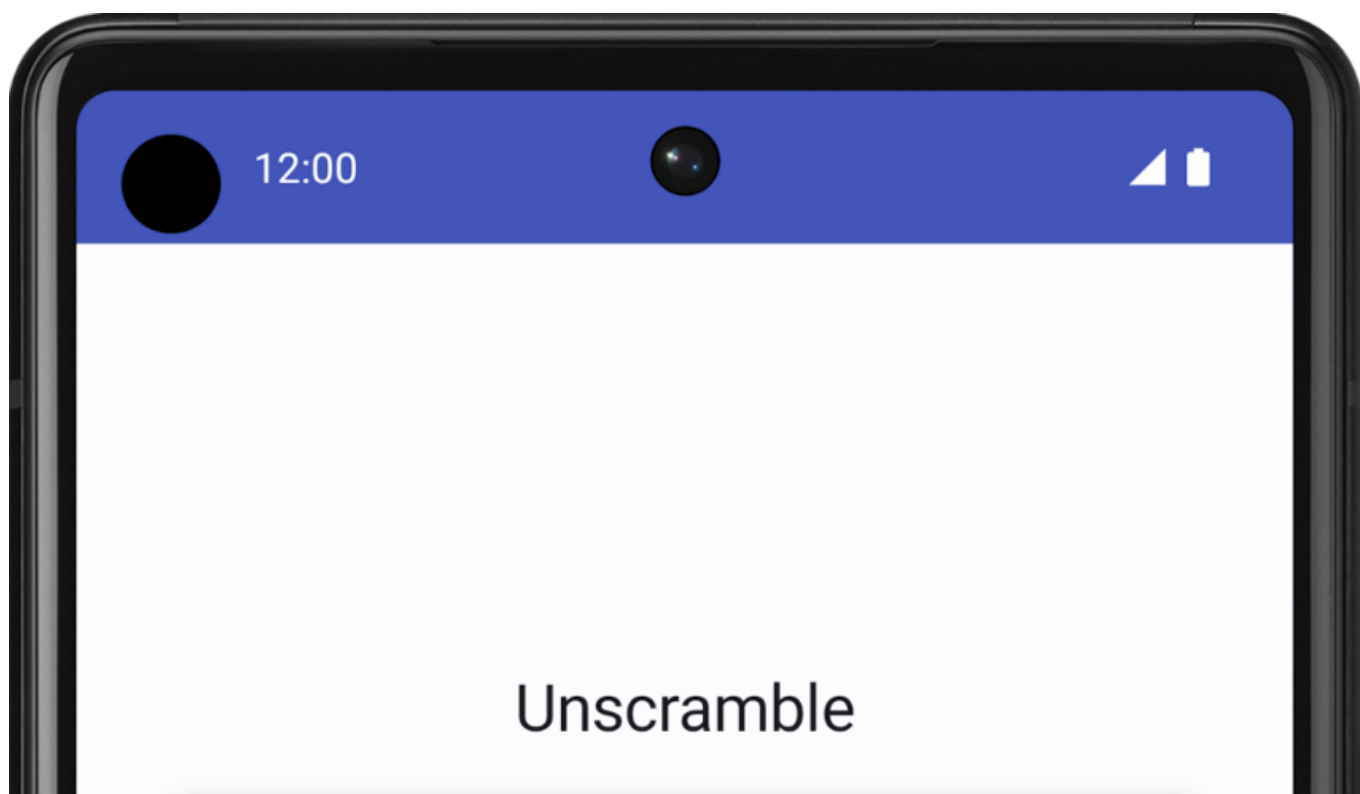
В предыдущих уроках вы узнали об изменении конфигурации в Android. Когда происходит изменение конфигурации, Android перезапускает активность с нуля, запуская все обратные вызовы запуска жизненного цикла.

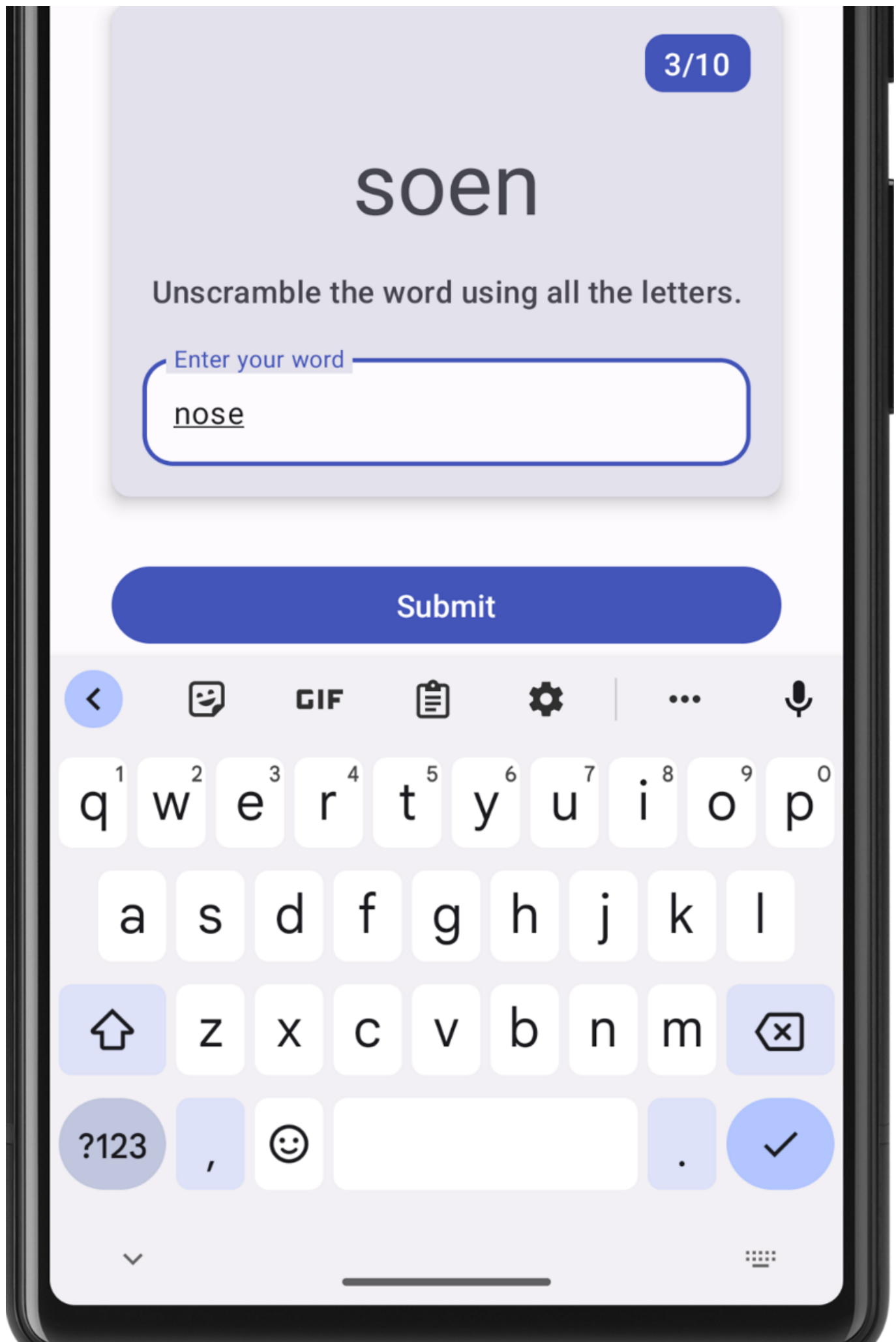
Модель ViewModel хранит данные, связанные с приложением, которые не уничтожаются, когда фреймворк Android уничтожает и воссоздает активность. Объекты ViewModel автоматически сохраняются и не уничтожаются, как экземпляры активности, при изменении конфигурации. Данные, которые они хранят, сразу же становятся доступны после рекомпозиции.

В этом задании вы проверите, сохраняет ли приложение состояние пользовательского интерфейса при изменении конфигурации.

- Запустите приложение и сыграйте несколько слов. - Измените конфигурацию устройства с книжной на альбомную или наоборот.

Обратите внимание, что данные, сохраненные в пользовательском интерфейсе состояния ViewModel, сохраняются при изменении конфигурации.







## Заключение

---

Теперь вы понимаете, как руководство по архитектуре приложений для Android рекомендует разделять классы с разными обязанностями и управлять пользовательским интерфейсом из модели.